

Post-Training and Alignment: Crash Pack

This is the compressed volume for Role Track 2. It assumes you have read the base track chapters. It keeps only the five things you must recall cold: decision rules, numbers, formulas, traps, and reps. Five parts, in the order you use them on the job.

How to use this pack.

- Read Part A when you face a choice: which algorithm, which mix, when to stop.
- Memorize Part B like multiplication tables. Every formula has units and a worked number.
- Read Part C before every training run. Part C lists the traps in the order they will find you.
- Do Part D out loud, one question at a time, with the answers covered.
- Print Part E. Tape it to the wall next to the training dashboard.

Part A: Decision rules

The rules use if-then form because that is how you use them at 11pm when a run is red.

A.0 The ladder

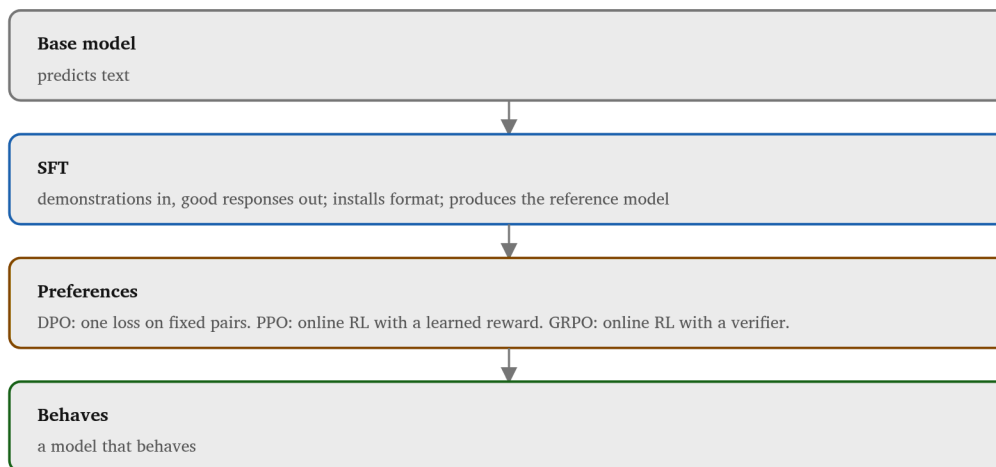
Post-training starts from a base model that predicts text and ends with a model that behaves. Three rungs, in order. Every rule below assumes them.

1. **SFT first.** Supervised fine-tuning trains the model on demonstration examples: prompt in, good response out, next-token loss. This installs the format and the starting behavior. It also produces the reference model: a frozen copy of the trained policy. The policy is the model you train and the reference is the anchor it must not drift too far from. You measure every later stage against that anchor.
2. **Then preferences.** Three algorithms, one job: make the policy prefer better responses.
 - **DPO** (Direct Preference Optimization) is one classification loss on fixed preference pairs, with no RL loop.
 - **PPO** (Proximal Policy Optimization) is online RL. The policy generates its own outputs, called rollouts. A learned reward model scores them. The policy updates against those scores.
 - **GRPO** (Group Relative Policy Optimization) is online RL with a verifier instead of a learned reward. Sample G responses per prompt. Score them 1/0. Learn from each response's standing relative to its group.
 - **RLHF** (reinforcement learning from human feedback) is the family name for the SFT, reward model, PPO pipeline.

3. **Everything under one leash.** Every stage optimizes $J(\pi) = E[r] - \beta \cdot \text{KL}(\pi || \pi_{\text{ref}})$. That is mean reward minus β times the KL (Kullback-Leibler) divergence. The KL measures how far the policy's token distribution has drifted from the reference. β prices drift. On-policy data means data from the current policy; off-policy means data from older or other models, which misranks what the policy actually produces.

First decision: which algorithm. The choice shapes everything downstream: the data you collect, the compute you need, and the failure modes you watch.

The post-training ladder: base model to behaving model

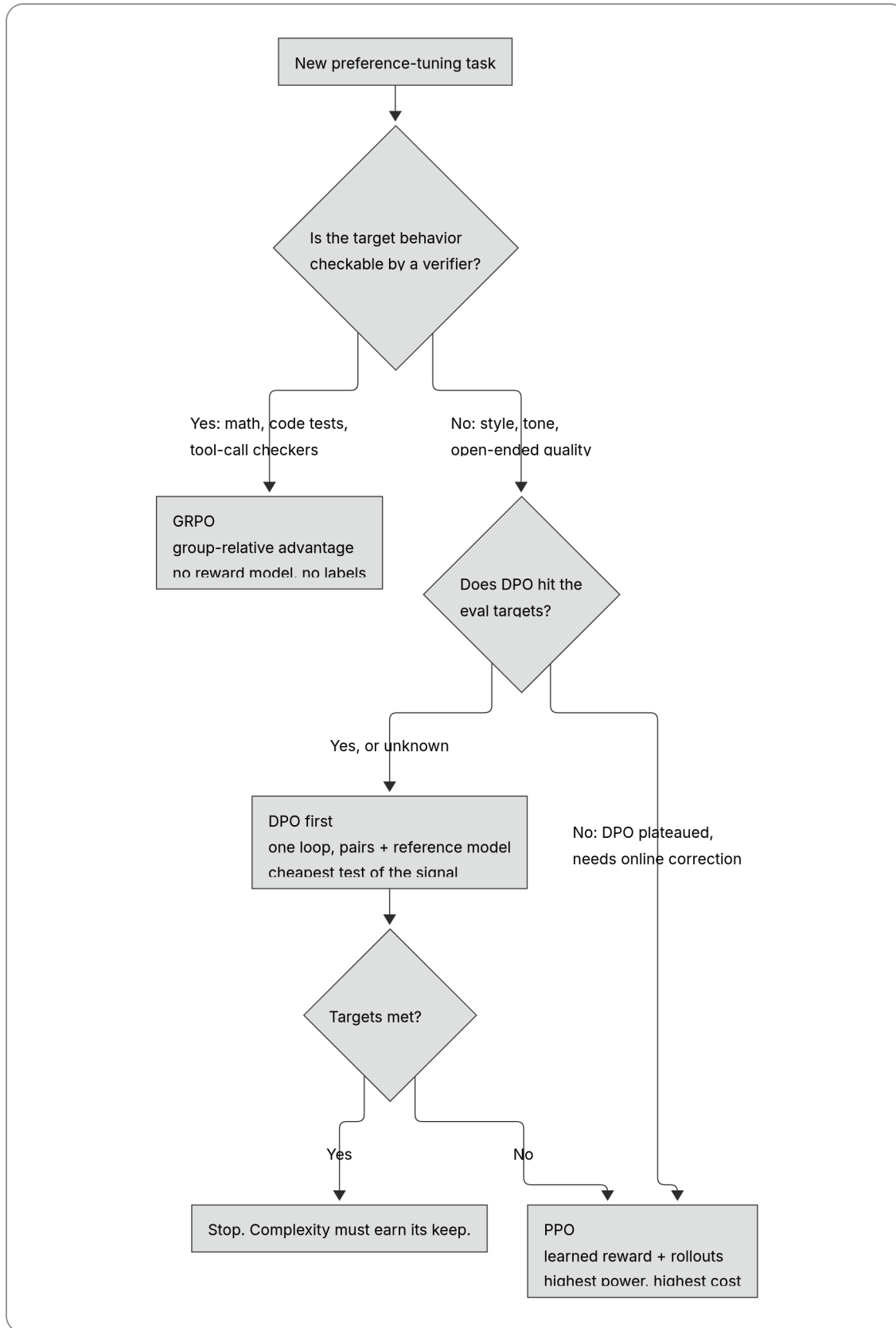


One leash at every stage: $J(p) = E[r] - \beta \times \text{KL}(p || p_{\text{ref}})$. Beta prices drift.

SFT produces the reference: a frozen copy of the trained policy that every later stage is measured against.

Figure A.0. The post-training ladder: base model to behaving model. SFT installs the format and produces the reference model; the preference stage (DPO, PPO, or GRPO) is where behavior is actually shaped.

A.1 DPO vs PPO vs GRPO: the selection rules



How to read this diagram

1. The first branch is the only one that matters most days. A verifier (an automatic checker with right and wrong answers, like unit tests or a math answer key) turns RL into supervised-like signal. No verifier means you need human taste, and human taste means a reward model or pairs.
2. DPO is the default first try even when you suspect you will need PPO. If DPO wins, you saved a month of infra.
3. The loop back from DPO to PPO is the standard production playbook: SFT (supervised fine-tuning: training on demonstration examples), DPO baseline, then escalate only on evidence.
4. GRPO never feeds back into PPO in this chart. If you have a verifier, you do not need a learned reward.
5. GRPO's "group-relative advantage" works like this. Sample a group of completions per prompt. Score each one against the group average. Advantage means how much better than average this completion was. You need no separate value model.

The signal table, one line per algorithm.

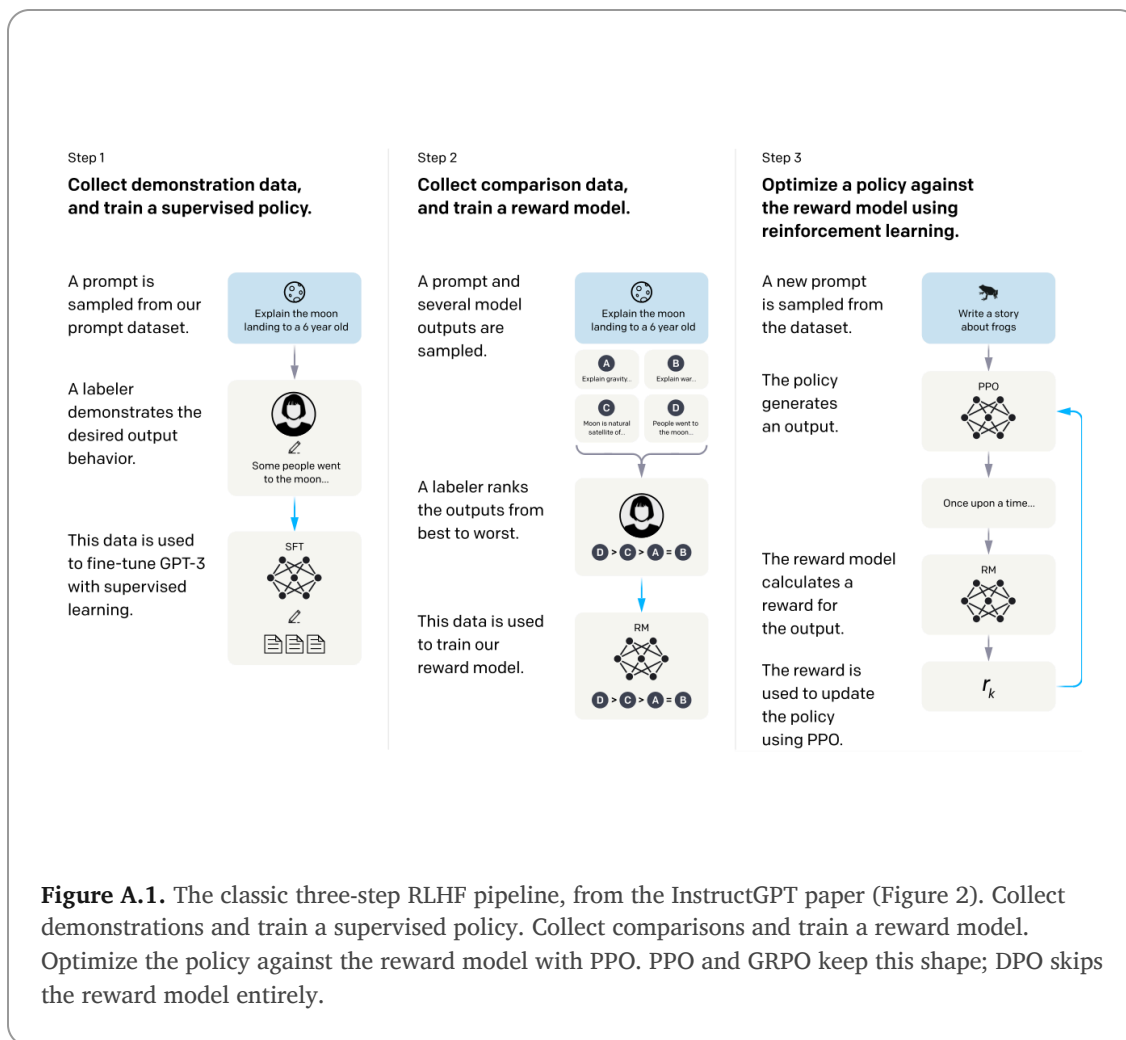


Figure A.1. The classic three-step RLHF pipeline, from the InstructGPT paper (Figure 2). Collect demonstrations and train a supervised policy. Collect comparisons and train a reward model. Optimize the policy against the reward model with PPO. PPO and GRPO keep this shape; DPO skips the reward model entirely.

	PPO	DPO	GRPO
Signal	learned reward model	preference pairs	verifier scores
Online sampling	yes	no	yes, groups per prompt
Extra models	reward model + value head (a small network predicting expected reward, used as the training baseline)	reference model only	verifier only
Infra cost	highest	lowest	medium, generation-heavy
Stability	fiddly, needs babysitting	stable, simple	stable if the verifier is good
Best for	open-ended quality	quick preference tuning	math, code, reasoning
Classic failure	KL spike, value collapse	distribution shift (the policy moving into regions the training data never covered), rejected-only gaming	verifier gaming, uniform groups

Six selection rules.

1. If a verifier exists, use GRPO. Exact rewards beat learned rewards every time.
2. If no verifier exists, run DPO first. One training loop, no sampling, no reward model.
3. Escalate to PPO only when DPO plateaus below target on open-ended behavior. PPO buys online correction of distribution shift at 3 to 5 times the complexity.
4. Never start with PPO when the reward model has not trained on the current policy's outputs. Off-policy reward models misrank on-policy samples, and PPO will optimize the misranking.
5. Filter GRPO prompts the model always gets right or always gets wrong. Uniform groups have zero standard deviation and contribute no gradient.
6. Watch DPO's chosen log-probs (the log-probabilities the model assigns to each token of the preferred response) during training. If the loss falls while chosen log-probs fall, the model games the loss by pushing down rejected responses only. That checkpoint is poison.

Filter GRPO prompts: keep only mixed groups

Advantage = $(r - \text{mean}) / \text{std}$. If $\text{std} = 0$ the advantage divides by zero.

Group A: [1, 1, 1, 1]

all G responses right
mean = 1.0, std = 0

SKIP: no signal

division by zero

Group B: [1, 0, 1, 0]

mixed success
mean = 0.5, std = 0.5

KEEP: trains

advantages +1, -1, +1, -1

Group C: [0, 0, 0, 0]

all G responses wrong
mean = 0.0, std = 0

SKIP: no signal

division by zero

Train only on mixed groups

A.1 rule 5: filter always-right and always-wrong prompts

Uniform groups carry no relative information. The code guard is $\text{std} < \text{epsilon}$: skip.

Figure A.6. GRPO prompt filtering. Groups that the model always gets right or always gets wrong have zero standard deviation. Their advantages divide by zero, so they carry no signal. Train only on mixed groups.

You have chosen the algorithm. Now the shared enemy of all three: once the reward model becomes the target, it stops measuring what you wanted.

A.2 Reward model overoptimization: the response rules

Goodhart's law runs the show: once the reward model becomes the target, it stops measuring what you wanted. The proxy score keeps climbing while true quality peaks and falls. These rules are the response.

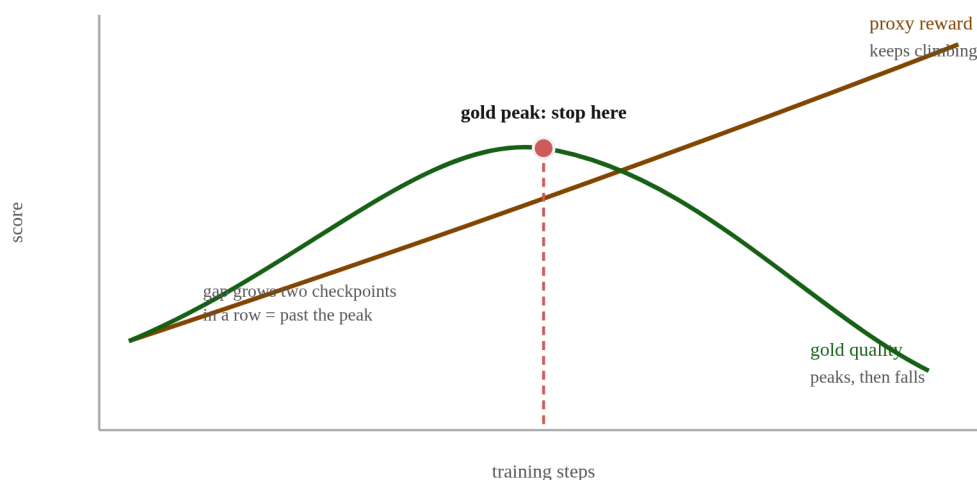
Overoptimization: proxy reward vs gold quality

Figure A.2. The overoptimization signature. The proxy (what you train against) climbs the whole run. The gold judge (what you actually want) peaks, then falls as the policy exploits the proxy's mistakes. Early-stop at the gold peak, never on the proxy. If the gap grows for two checkpoints in a row, the run is past the peak.

1. Keep a gold judge that never trains the policy: a held-out reward model, a bigger judge, or human labels. Score every checkpoint with both proxy and gold.
2. If the gold-proxy gap grows for two checkpoints in a row, stop the run. You are past the peak.
3. Early-stop on the gold metric, never on the proxy. The shipped checkpoint is the gold peak, not the final step.
4. If KL from the SFT model spikes while human-judged quality is flat, pause and inspect. That is the classic signature, not a hyperparameter to tune through.
5. Read 50 outputs at the current checkpoint before every ship review. Overoptimized models have tells: sycophancy (agreeing with the user regardless of the truth), hedging stacks (layers of qualifiers that avoid saying anything), listicles for everything, repeating the prompt back.
6. When overoptimization appears early (small KL), the reward model is too easy to game. Answers: retrain it on fresh on-policy pairs, ensemble it, or add a KL penalty increase. Do not just train longer.
7. Keep a fixed set of reward-hacking probes. Score them every run. Rising proxy plus rising hack rate is not progress.

You have chosen the algorithm and you watch the proxy. Next, the data mix the algorithms eat: how much safety, how much capability, and what happens when the ratio is wrong.

A.3 Data-mix ratios: the numbers that hold

1. Safety data stays at 5 to 15 percent of the SFT mix. Above that you buy safety with capability. Tune inside the range by watching false-refusal rates.
2. Pair every refusal with a nearby compliance. For each disallowed prompt, include a benign prompt on an adjacent topic with a full helpful answer. The model learns the boundary, not just the refusal.
3. Never run a safety-only training stage. Mix capability examples into every safety batch, or the model forgets what it was kept safe for.
4. Sample 2 to 4 responses per prompt for preference labeling. Recipe that works: 2 from the current policy, 1 from a stronger model, 1 from an older checkpoint.
5. Refresh preference data every few RL rounds with samples from the current policy. Reward models go stale as the policy moves.
6. For rejection sampling, generate $k = 8$ to 64 completions per prompt at moderate to high temperature (the sampling knob: higher temperature means more random outputs), keep the ones the verifier passes.
7. Oversample close calls in preference pairs. Easy pairs contribute near-zero gradient. Hard pairs teach the boundary.
8. Quarantine pairs below 60 percent three-way labeler agreement. They are ambiguous prompts or broken rubrics, not training data.
9. Near-dedup threshold near 0.8 Jaccard similarity (size of the overlap divided by size of the union) on shingles (short overlapping text windows). Tune it on 200 hand-labeled pairs before running it over millions of prompts.
10. Version every dataset with a hash and a changelog. When a run behaves oddly, the first question is what changed in the data.

All three algorithms from A.1 optimize one shared objective with a leash. This section sizes the leash: how far the policy may drift, and what the drift costs.

Data-mix ratios: the numbers that hold

Safety fraction above that you buy safety with capability	5-15% of the SFT mix
Refusal pairs benign prompt, adjacent topic, full answer: the model learns the boundary	every refusal + a nearby compliance
Never safety-only or the model forgets what it was kept safe for	capability examples in every safety batch
Preference sampling recipe: 2 from current policy, 1 from a stronger model, 1 from an older checkpoint	2-4 responses per prompt
Refresh reward models go stale as the policy moves	new preference data every few RL rounds
Rejection sampling moderate to high temperature	k = 8-64 completions per prompt

Figure A.3. Data-mix ratios: the numbers that hold. Safety at 5 to 15 percent of the SFT mix. Every refusal paired with a nearby compliance. Never safety-only. 2 to 4 responses per prompt in the 2-1-1 recipe.

A.4 KL budgets: the leash rules

The shared objective behind all three algorithms:

$$J(\pi) = E[r(x, y)] - \beta \cdot \text{KL}(\pi \parallel \pi_{\text{ref}})$$

Where: the policy π is the one you train. The reference model π_{ref} is usually the SFT checkpoint. The reward $r(x, y)$ scores response y to prompt x . The coefficient β (beta) prices drift: every unit of KL away from the reference costs β units of reward.

Beta prices drift. Every unit of KL away from the reference costs beta units of reward.

1. Log KL from the reference per training step, not per epoch (one full pass over the training data). A spike is visible in steps and invisible in epochs.
2. KL climbing steadily with rising gold scores is healthy optimization. KL spiking vertically is a broken run: beta too small, learning rate too high, or a reward bug.
3. KL flat from step one means the run learned nothing. Check the advantage signal (how much better each response was than expected) and the data before touching hyperparameters.
4. DPO's beta is the whole KL control: typical working value 0.1. Lower beta means stronger preference signal and more drift. Higher beta means safer and weaker.
5. In PPO, beta and the clip (epsilon = 0.2 typical) are two leashes at different scales. The clip bounds each step. Beta bounds the total drift.
6. Never remove the KL term to "see what happens." What happens: the policy discovers reward-model blind spots. The outputs turn to gibberish the proxy loves.

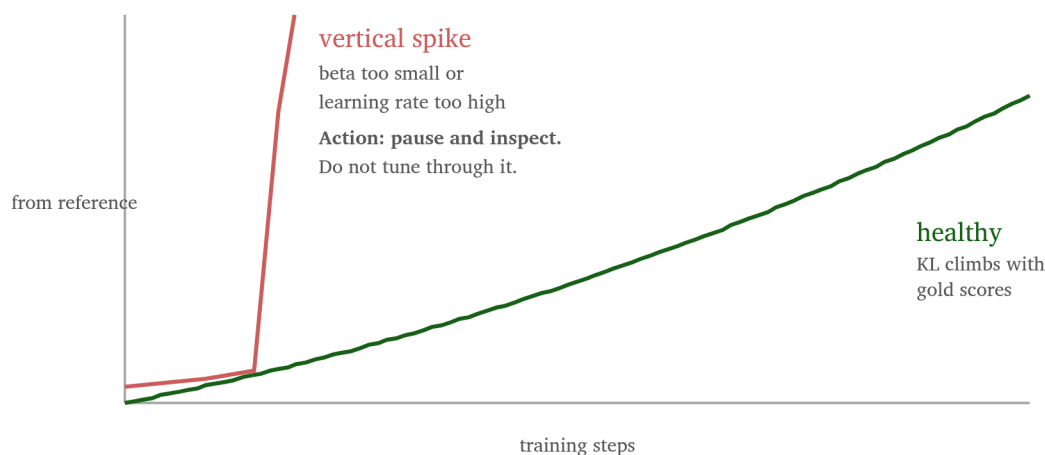
KL from the reference: healthy climb vs spike

Figure A.7. KL from the reference over training steps. A steady climb with rising gold scores is healthy optimization. A vertical spike means the leash failed: beta is too small or the learning rate is too high. Pause and inspect. Do not tune through a spike.

You have trained the model. The last decision is the ship decision: four gate layers, in order. Each layer answers whether the model is ready for the next one.

KL budgets: the leash rules

$$J(p) = E[r(x,y)] - b \times KL(p || p_{ref})$$

beta prices drift: every unit of KL away from the reference costs beta units of reward

Log KL from the reference per step, not per epoch.

A spike is visible in steps and invisible in epochs.

- █ **Climbing steadily + gold scores rising**
healthy optimization
- █ **Spiking vertically**
broken run: beta too small, LR too high, or a reward bug
- █ **Flat from step one**
the run learned nothing: check the advantage signal

Figure A.4. KL budgets: the leash rules. One objective at every stage: $J(p_i) = E[r] - \text{beta} \times KL(p_i || p_{ref})$. Log KL per step, not per epoch; a spike is visible in steps and invisible in epochs.

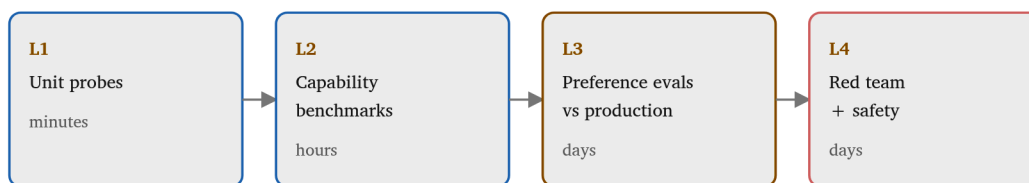
A.5 Eval gates: the ship rules

1. Four layers, in order: unit probes (minutes), capability benchmarks (hours), preference evals vs production (days), red team and safety (days). A failure at any layer blocks the release.

2. Never trust a win-rate delta without a confidence interval. A 54 percent win rate on 200 judgments has a standard error near 3.5 points. That win may be noise.
3. Use the Wilson interval for binomial rates, especially at small n or near 0 and 1. The normal approximation lies at the edges.
4. Slice results by prompt category before celebrating. The aggregate hides regressions. A candidate that wins overall but loses on code has a code problem.
5. Benchmarks are guardrails, not targets. The new checkpoint must stay within noise of the current best on every suite. A 2-point math gain with a 3-point instruction-following drop is a regression.
6. Fresh prompts for preference evals, every cycle. Reusing prompts optimizes for the test set.
7. Track the evals themselves. Benchmarks saturate, autoraters drift, human judges recalibrate. Version eval sets and re-baseline on a schedule.
8. Refusal is two numbers, not one. Track true-refusal rate on the disallowed set and false-refusal rate on the benign-edge set, at every checkpoint.
9. License every autorater (an LLM used as a judge) before its scores count: calibrate against humans, report Cohen's kappa (a chance-corrected agreement score between the autorater and a human), compare against the human-human ceiling.
10. When a gate lacks the power to detect your effect size, say so in the review. A gate that cannot detect the effect size you care about is theater. Size it with the standard-error formula first.

Next: Part B gives the formulas behind the rules: Bradley-Terry, the DPO loss, the KL leash, and the statistics that keep gates honest.

Eval gates: the ship rules



A failure at any layer blocks the release.

54% win rate on 200 judgments: SE near 3.5 points. That win may be noise.

Use the Wilson interval for binomial rates. Slice by prompt category: the aggregate hides regressions.

Figure A.5. Eval gates: the ship rules. Four layers, in order, minutes to days. A failure at any layer blocks the release. 54 percent on 200 judgments has a standard error near 3.5 points: that win may be noise.

Part B: Numbers and formulas (memorize these)

Part A gave you if-then rules for post-training. Every rule rests on a formula: the preference probability behind DPO, the KL budget behind the leash, the standard errors behind the gates. Part B is that math, each with a worked number. It answers the question Part A leaves open: what are the exact quantities the rules depend on?

Every entry has the formula, the units, what it means, and a worked number. If you cannot reproduce the worked number by hand, you do not know the formula.

B.1 Bradley-Terry: the preference probability

$$P(A \text{ beats } B) = \text{sigmoid}(r_A - r_B)$$

Where: r_A , r_B are the judge's scores (unitless logits) for the two responses; $\text{sigmoid}(x) = 1 / (1 + e^{-x})$. Output is a probability between 0 and 1.

Meaning: a judge prefers A over B with probability given by the sigmoid of the score gap. Only the difference matters. Add 100 to both scores and nothing changes. A score of 2.1 alone means nothing. It means "better than 0.4 on the same prompt."

The training loss is the negative log of that probability, summed over pairs:

$$L = -\sum \log \text{sigmoid}(r_{\text{chosen}} - r_{\text{rejected}})$$

Its gradient weights each pair by $(1 - p)$, the surprise. Pairs the model already ranks correctly contribute almost nothing. Hard pairs drive the update.

```
# Bradley-Terry probabilities from score gaps.
# WHAT:  converts reward-score differences into preference
#         probabilities, the exact mapping the reward-model loss
#         optimizes.
# WHY:   this is the number you read when a reward model says
#         "chosen scores 2.1, rejected scores 0.4". The gap, not
#         the values, is the information.
# BREAKS IF: you read raw scores as quality levels. A gap of 1.7
#             and a gap of 1.7 after adding 100 to both scores give
#             the same probability. Scale is free; differences rule.
import math

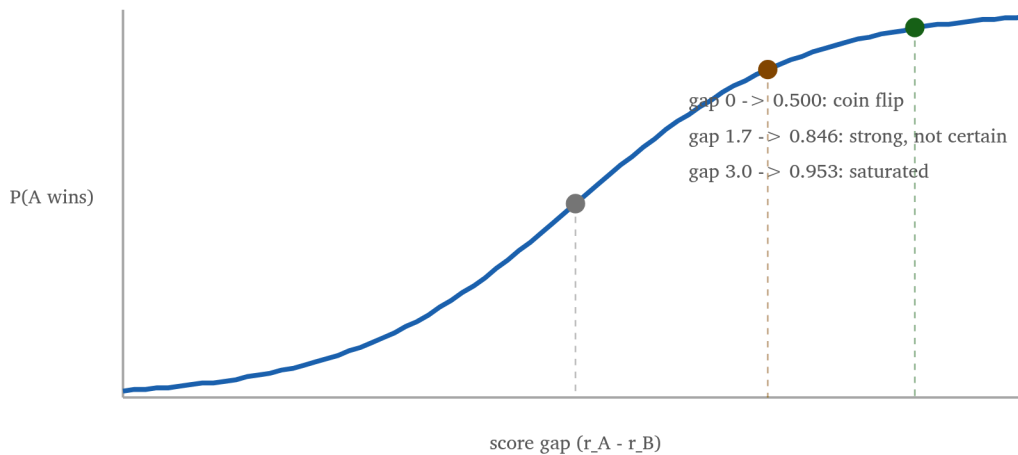
def pref_prob(r_a, r_b):
    # sigmoid of the gap: P(judge prefers A over B)
    return 1.0 / (1.0 + math.exp(-(r_a - r_b)))

for gap in [0.0, 0.5, 1.0, 1.7, 3.0]:
    print(f"gap {gap}>4: P = {pref_prob(gap, 0.0):.3f}")
```

How to read this diagram

1. A gap of 0 is a coin flip: 0.500. The model has no opinion.
2. A gap of 1.7 (the 2.1 vs 0.4 example) gives 0.846. Strong but not certain preference.
3. A gap of 3.0 gives 0.953. Past a gap of about 3, the sigmoid saturates and bigger gaps barely change the probability. That saturation is why reward scores drift in scale during training without changing behavior.

Bradley-Terry is the preference model. DPO is Bradley-Terry with the reward model solved out of the RLHF objective, so the policy trains directly on preference pairs.

Bradley-Terry: the sigmoid of the score gap

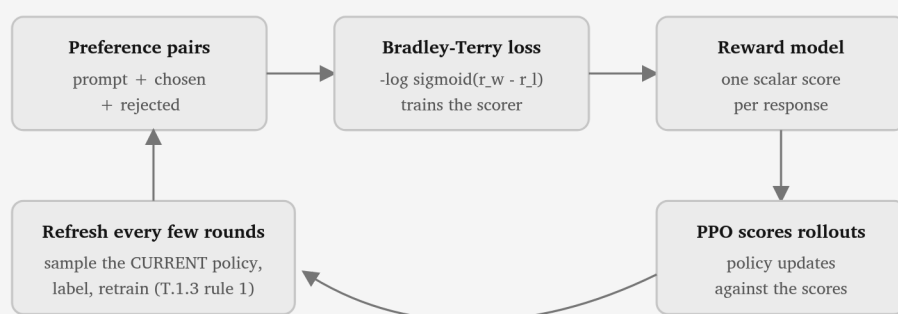
Only the gap matters: add 100 to both scores and nothing changes.

Loss = $-\log P$. Gradient weight = $(1 - p)$: hard pairs drive the update, easy pairs barely teach.

Figure B.1. Bradley-Terry: the sigmoid of the score gap. Only the gap between scores matters, never their absolute level.

How to read it.

1. Find the gap between the two scores on the bottom axis, then read up to the curve.
2. Gap 0: 0.500, a coin flip, no opinion. Gap 1.7 (the 2.1 vs 0.4 example): 0.846, strong but not certain.
3. Past a gap of about 3 the curve saturates. Bigger gaps barely move the probability, so training gets no gradient from them.
4. The loss is $-\log P$, and the gradient weight is $(1 - p)$: hard pairs drive the update, easy pairs barely teach.

Reward model training and refresh loop

A stale reward model misranks the current policy. PPO then optimizes the misranking.
The reward model must chase the policy, not the other way around.

Figure B.11. Reward model training and refresh loop. Preference pairs train the reward model through the Bradley-Terry loss. The model scores rollouts for PPO. Every few rounds, fresh samples from the current policy refresh the model.

How to read this diagram

1. Preference pairs enter on the left: one prompt, one chosen response, one rejected response.
2. The Bradley-Terry loss converts each pair into a gradient: minus log sigmoid of the score gap.
3. The trained reward model emits one scalar score per response. What it is: a learned judge. Why it exists: PPO needs a fast scorer for rollouts. What breaks if it fails: a bad scorer teaches PPO the wrong ranking.
4. PPO generates rollouts from the current policy and updates the policy against the reward model scores.
5. The gold loop at the bottom is the refresh (T.1.3 rule 1): sample the current policy, label fresh pairs, retrain. Why it exists: the reward model goes stale as the policy drifts. What breaks if it fails: a stale model misranks on-policy samples and PPO optimizes the misranking.

Bradley-Terry, worked by hand (from the example above)

```

chosen score 2.1, rejected score 0.4
gap = 2.1 - 0.4 = 1.7
P(chosen wins) = sigmoid(1.7) = 1 / (1 + e^-1.7) = 0.846
loss = -log(0.846) = 0.168 nats (natural-log units of information)
gradient weight = 1 - p = 0.154

easy pair: gap 4.0 -> p = 0.982 -> weight 0.018 (barely updates)
hard pair: gap 0.2 -> p = 0.550 -> weight 0.450 (drives the update)

only the gap matters: add 100 to both scores, nothing changes

```

B.2 DPO loss: the worked number

$$L_{\text{DPO}} = -\log \text{sigmoid}(\beta \cdot [\log(\pi(\text{chosen})/\pi_{\text{ref}}(\text{chosen})) - \log(\pi(\text{rejected})/\pi_{\text{ref}}(\text{rejected}))])$$

Where: π is the policy, π_{ref} the reference model, β (beta) is unitless with typical value 0.1, and the loss is in nats (natural-log units).

Meaning: DPO is Bradley-Terry with the reward model solved out of the RLHF objective. The implicit reward of a response is $\beta \cdot \log(\pi(y|x) / \pi_{\text{ref}}(y|x))$: how much more the policy likes it than the reference does.

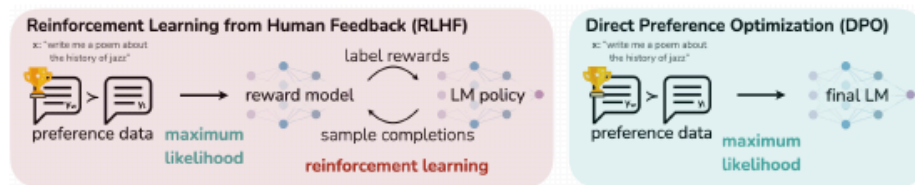


Figure 1: **DPO optimizes for human preferences while avoiding reinforcement learning.** Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an *implicit* reward model whose corresponding optimal policy can be extracted in closed form.

we will show that the RL-based objective used by existing methods can be optimized exactly with a simple binary cross-entropy objective, greatly simplifying the preference learning pipeline.

At a high level, existing methods instill the desired behaviors into a language model using curated sets of human preferences representing the types of behaviors that humans find safe and helpful. This preference learning stage occurs after an initial stage of large-scale unsupervised pre-training on a large text dataset. While the most straightforward approach to preference learning is supervised fine-tuning on human demonstrations of high quality responses, the most successful class of methods is reinforcement learning from human (or AI) feedback (RLHF/RLAIF; [12][2]). RLHF methods fit a reward model to a dataset of human preferences and then use RL to optimize a language model

Figure B.2. DPO vs RLHF, from the DPO paper (Figure 1). RLHF fits a reward model to preferences, then runs RL against it (two phases, four models in memory). DPO trains the policy directly on the same preference pairs with one classification loss. DPO solves out the reward model instead of approximating it.

Worked example. Beta = 0.1. Policy log-probs: chosen -2.0, rejected -3.5. Reference log-probs: chosen -2.2, rejected -2.8.

```

# DPO loss, worked by hand.
# WHAT: computes the DPO loss for one preference pair from the
#       four log-probs, exactly as the formula defines it.
# WHY:  this is the number the DPO training loop minimizes. If
#       you can compute it by hand, you can read a DPO loss curve
#       and know what the policy is actually doing.
# BREAKS IF: you forget the reference terms. Without pi_ref the
#            loss has no anchor, and the policy can minimize it by
#            pushing down rejected log-probs forever while chosen
#            log-probs fall too. That is the rejected-only gaming
#            failure in A.1 rule 6.
import math

beta = 0.1
lp_chosen, lp_rejected = -2.0, -3.5      # policy log-probs
lr_chosen, lr_rejected = -2.2, -2.8      # reference log-probs

margin_chosen = lp_chosen - lr_chosen     # 0.2: policy likes chosen a bit
more than ref
margin_rejected = lp_rejected - lr_rejected # -0.7: policy likes rejected less
than ref
gap = margin_chosen - margin_rejected     # 0.9
z = beta * gap                           # 0.09
p = 1.0 / (1.0 + math.exp(-z))           # sigmoid(0.09)
loss = -math.log(p)
print(f"gap={gap:.1f} beta*gap={z:.2f} sigmoid={p:.3f} loss={loss:.3f}")
print(f"gradient weight (1-p) = {1 - p:.3f}: this pair still teaches")

```

How to read this diagram

1. The chosen margin is 0.2 and the rejected margin is -0.7, so the gap is 0.9. The policy already ranks this pair correctly.
2. Times beta = 0.1, the logit (the raw score fed to the sigmoid) is 0.09. Sigmoid gives 0.522, so the loss is $-\log(0.522) = 0.649$.
3. The gradient weight $(1 - 0.522) = 0.478$. Same $(1 - p)$ shape as the Bradley-Terry gradient in B.1, because DPO is Bradley-Terry with the reward solved out. Pairs the model gets right contribute less.

Memorize: beta = 0.1 is the standard starting point. The loss falls when the gap grows, and the gap grows by raising chosen log-probs, lowering rejected log-probs, or both. Only the first is real progress.

DPO needs no RL and no reward model. GRPO keeps RL but drops the learned value model: the group of completions per prompt becomes its own baseline.

DPO loss, worked by hand (beta = 0.1)

```
policy log-probs:   chosen -2.0   rejected -3.5
reference log-probs: chosen -2.2   rejected -2.8
```

```
implicit reward, chosen  = 0.1 x (-2.0 - (-2.2)) = 0.020
implicit reward, rejected = 0.1 x (-3.5 - (-2.8)) = -0.070
gap = 0.020 - (-0.070) = 0.090
P = sigmoid(0.090) = 0.5225
loss = -log(0.5225) = 0.649 nats
```

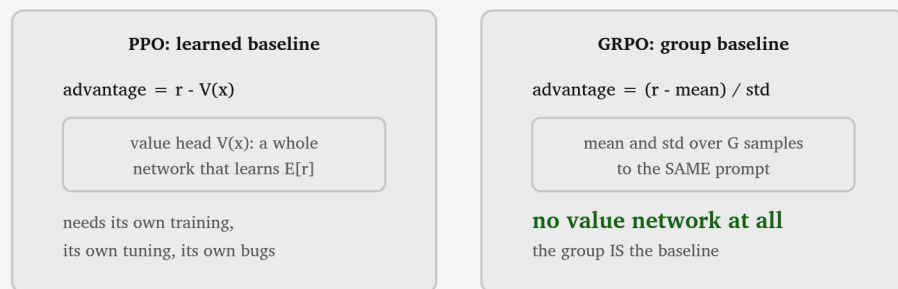
the policy already likes the chosen answer more than the reference does, so the loss is small: the preference is nearly satisfied before training even starts

B.3 GRPO advantage: the worked numbers

$$A_i = (r_i - \text{mean}(r)) / \text{std}(r)$$

Where: r_i is the verifier score (usually 0 or 1) for completion i in the group. Take mean and std over the group's completions for one prompt. Advantages are unitless, in standard-deviation units.

Meaning: score each response in the group against its siblings, not against a learned baseline. You need no value network. The group is its own baseline.

Advantage baselines: PPO vs GRPO

Same job: center the reward so the update sees surprise, not raw score.

GRPO pays for it in generation (G samples per prompt) instead of in a second network.

Figure B.12. Advantage baselines: PPO vs GRPO. PPO centers rewards with a learned value head, a whole second network. GRPO centers with the group mean and standard deviation. The group replaces the value network.

How to read this diagram

1. Left: PPO. Advantage equals reward minus the value head estimate of expected reward.
2. The value head is a second network. What it costs: its own training, its own tuning, its own failure modes.
3. Right: GRPO. Advantage equals $(r \text{ minus group mean})$ divided by group standard deviation, computed over G samples for the same prompt.
4. The group is its own baseline, so no value network exists. What GRPO pays instead: generation cost, G samples per prompt, which is why Appendix 7A exists.

```

# GRPO group-relative advantages, three cases.
# WHAT:  standardizes verifier scores within one prompt's group.
# WHY:   this is the entire GRPO advantage estimator. Three cases
#         show everything: clean split, one standout, uniform group.
# BREAKS IF: the group standard deviation is zero. Uniform groups
#             (all right or all wrong) carry no signal and must be
#             skipped, or the division blows up.
def grpo_advantages(rewards):
    # rewards: verifier scores for G responses to ONE prompt
    n = len(rewards)
    mean = sum(rewards) / n
    var = sum((r - mean) ** 2 for r in rewards) / n
    std = var ** 0.5
    if std < 1e-9:
        return None # uniform group: no signal
    return [round((r - mean) / std, 2) for r in rewards]

print("split [1,1,0,0]:", grpo_advantages([1, 1, 0, 0]))
print("standout [1,0,0,0]:", grpo_advantages([1, 0, 0, 0]))
print("uniform [1,1,1,1]:", grpo_advantages([1, 1, 1, 1]))

```

How to read this diagram

1. Split group [1,1,0,0]: mean 0.5, std 0.5, advantages [+1, +1, -1, -1]. Correct solutions pushed up equally, wrong ones pushed down equally.
2. Standout [1,0,0,0]: mean 0.25, std about 0.43, advantages [+1.73, -0.58, -0.58, -0.58]. The single correct solution gets a big push because it stood out.
3. Uniform [1,1,1,1]: std is 0, no signal. Filter these prompts out of the GRPO batch.

Memorize: the standout case is why GRPO works on hard reasoning. Rare correct solutions get amplified, not averaged away.

GRPO drops the critic. PPO keeps it, and adds a different leash: clipping, which holds each update inside a trust region around the old policy.

GRPO group-relative advantages: three cases

clean split

rewards: 1 1 0 0

advantages:

+1.00 +1.00 -1.00 -1.00

one standout

rewards: 1 0 0 0

advantages:

+1.73 -0.58 -0.58 -0.58

uniform group

rewards: 1 1 1 1

advantages:

no signal: skip

$A = (r - \text{group mean}) / \text{group std}$. The group is its own baseline: no value network needed.

Uniform groups carry no signal and must be skipped, or the division blows up.

Figure B.3. GRPO group-relative advantages: three worked cases. The group is its own baseline, so you need no value network. Uniform groups carry no signal. Skip them, or the division blows up.

GRPO advantage, worked by hand: rewards [1, 1, 0, 0]

mean = 0.5, std = 0.5

$A = (r - \text{mean}) / \text{std} = [+1.0, +1.0, -1.0, -1.0]$

the two correct answers get pushed up, the two wrong ones get pushed down, and no value network was needed:
the group is its own baseline

uniform group [1, 1, 1, 1]: std = 0, no signal, skip it

B.4 PPO clip: the trust region per step

$$L = \min(\text{ratio} \cdot A, \text{clip}(\text{ratio}, 1 - \text{eps}, 1 + \text{eps}) \cdot A)$$

Where: $\text{ratio} = \pi_{\text{new}} / \pi_{\text{old}}$ is unitless and starts at 1.0. A is the advantage in reward units. Eps is unitless with typical value 0.2.

Meaning: when the advantage is positive, the update can push the probability ratio up to at most $1 + \text{eps} = 1.2$. One lucky sample cannot move the policy without bound. When the advantage is negative, the `min()` with the clipped value keeps the pessimistic bound. Stop pushing the policy once it has moved past $1 - \text{eps} = 0.8$.

```
# PPO clipped objective for one token, worked by hand.
# WHAT:  evaluates the min() of the unclipped and clipped
#        surrogate for a grid of ratios and advantages.
# WHY:   the clip is the whole stability story of PPO. Seeing the
#        numbers shows exactly where the leash bites.
# BREAKS IF: eps is set huge (the clip never binds and PPO is just
#             REINFORCE with extra steps) or the advantage estimate
#             is garbage (the clip bounds the step, not the direction).
def ppo_objective(ratio, advantage, eps=0.2):
    # min of unclipped and clipped: the pessimistic bound wins
    clipped = min(max(ratio, 1 - eps), 1 + eps)
    return min(ratio * advantage, clipped * advantage)

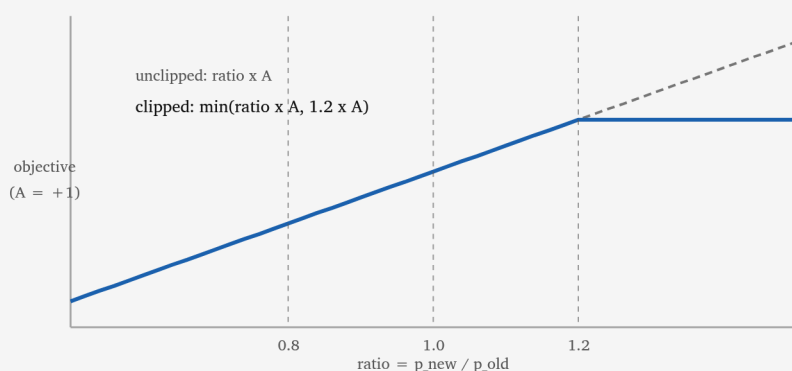
for ratio in [0.5, 0.9, 1.0, 1.1, 1.5, 3.0]:
    pos = ppo_objective(ratio, 1.0)    # good token: push up
    neg = ppo_objective(ratio, -1.0)   # bad token: push down
    print(f"ratio {ratio:>4}: A=+1 -> {pos:>6.2f}    A=-1 -> {neg:>6.2f}")
```

How to read this diagram

1. With $A = +1$, the objective grows with the ratio until 1.2, then flatlines. The policy is not rewarded for pushing past the trust region.
2. With $A = -1$, the objective improves as the ratio shrinks. But the `min()` keeps the pessimistic value: once past 0.8, further movement is not rewarded.
3. $\text{Eps} = 0.2$ is the standard value. Trust the ratio only in $[0.8, 1.2]$. That narrow band is what makes PPO stable with noisy reward models.

PPO's clip and DPO's beta both leash the policy. Measure the leash itself in KL divergence: how far the policy's token distribution has moved from the reference.

PPO clip: the trust region per step



$\text{eps} = 0.2$: the ratio is trusted in $[0.8, 1.2]$. Past 1.2 the objective goes flat.

One lucky sample cannot move the policy without bound. The clip bounds the step, not the direction.

Figure B.4. PPO clip: the trust region per step. With $\text{eps} = 0.2$, trust the ratio in $[0.8, 1.2]$. Past 1.2 the objective goes flat.

How to read it.

1. The dashed line is the unclipped objective, $\text{ratio} \times A$: it rises without bound as the policy moves.
2. The solid line is the clipped objective, $\min(\text{ratio} \times A, \text{clip}(\text{ratio}, 0.8, 1.2) \times A)$: it follows the dashed line inside the trust region, then flattens.
3. The vertical dashed lines at 0.8, 1.0, and 1.2 mark the trusted band. One lucky sample cannot move the policy without bound: the clip bounds the step, not the direction.

```
PPO clip, worked by hand (eps = 0.2, ratio = pi_new / pi_old)
```

```
advantage +1.0, ratio 1.5 (lucky sample pushes too far):
```

```
  unclipped = 1.5 x 1.0 = 1.5
```

```
  clipped   = 1.2 x 1.0 = 1.2
```

```
  L = min(1.5, 1.2) = 1.2  <- the leash binds
```

```
advantage -1.0, ratio 0.5 (policy moved too far down):
```

```
  unclipped = 0.5 x (-1.0) = -0.5
```

```
  clipped   = 0.8 x (-1.0) = -0.8
```

```
  L = min(-0.5, -0.8) = -0.8  <- pessimistic bound keeps it
```

```
inside [0.8, 1.2] the clip never binds: plain policy gradient
```

B.5 KL divergence: the distance that matters

$$\text{KL}(p \parallel q) = \sum_x p(x) \cdot \log(p(x) / q(x))$$

Where: p is the policy's token distribution, q is the reference distribution, and the sum runs over the vocabulary. Units: nats per token.

Meaning: how far the policy's token distribution has moved from the reference. This is the x-axis of the overoptimization plot and the leash in the RLHF objective. It is not symmetric: $\text{KL}(\pi_i \parallel \pi_{\text{ref}})$ measures the policy drifting away from the reference, which is the direction you care about.

In practice, estimate it per generated token as $\log \pi_i(\text{token}|\text{context}) - \log \pi_{\text{ref}}(\text{token}|\text{context})$, averaged over the rollout. Log it every step.

```
# KL divergence between two tiny distributions, by hand.
# WHAT:  the exact sum behind every KL number on the dashboard.
# WHY:   KL is the x-axis of the overoptimization plot (Part A.2)
#        and the leash in the RLHF objective. Knowing the sum lets
#        you read "KL = 3.2 nats" as "the policy has moved this far".
# BREAKS IF: you swap p and q. KL(p||q) weights by p: it measures
#             where the POLICY goes, using the reference only as the
#             yardstick. The reverse direction answers a question
#             nobody asked.
import math

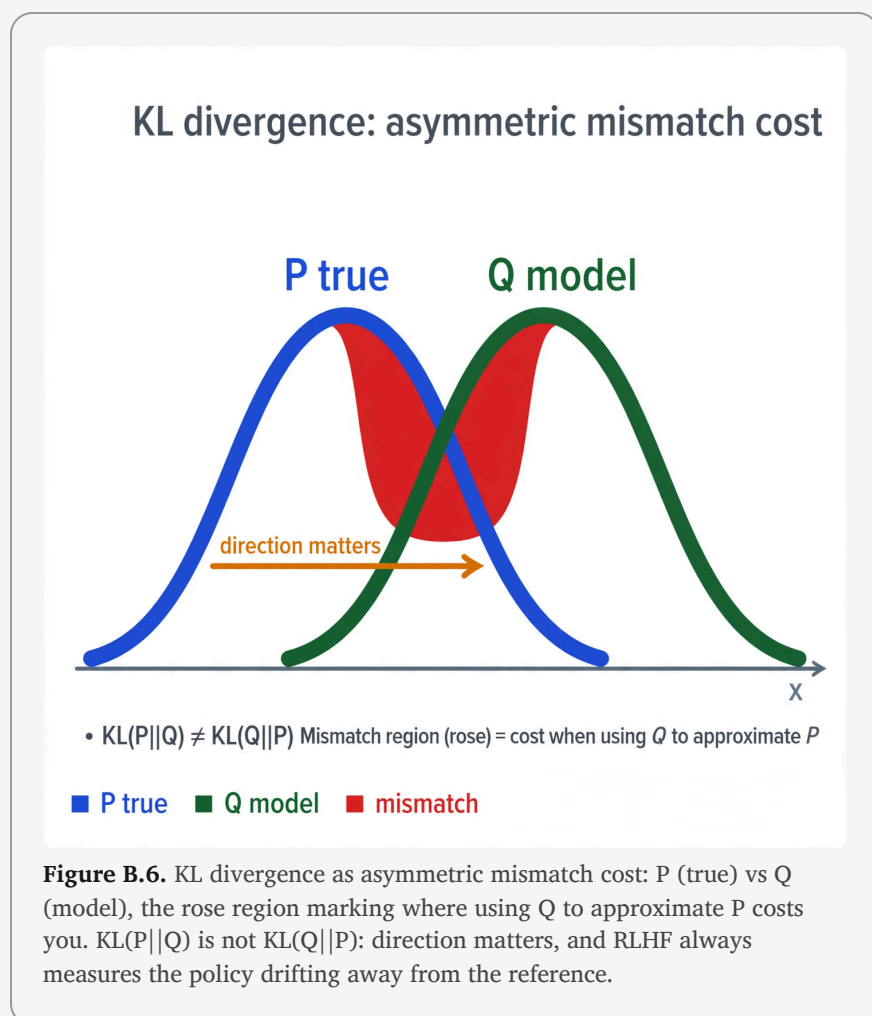
def kl(p, q):
    # p, q: lists of probabilities over the same outcomes
    return sum(pi * math.log(pi / qi) for pi, qi in zip(p, q))

print("identical:      ", round(kl([0.5, 0.3, 0.2], [0.5, 0.3, 0.2]),
4))
print("small drift:    ", round(kl([0.55, 0.28, 0.17], [0.5, 0.3, 0.2]),
4))
print("collapsed to one: ", round(kl([0.98, 0.01, 0.01], [0.5, 0.3, 0.2]),
4))
```

How to read this diagram

1. Identical distributions give exactly 0. KL is zero only when the policy matches the reference.
2. A small drift gives about 0.005 nats. Per-token KL numbers in real runs are small like this. They add up over a response.
3. Collapse to one token gives about 0.60 nats. When you see KL climbing toward whole nats per token, the policy has left the region where the reward model is trustworthy. That is the overoptimization x-axis.

KL measures drift. The next sections measure something else: whether your evals can see anything at all. Win rates are meaningless without error bars, so the statistics start here.

**B.6 Wilson interval: the honest confidence interval**

For a win rate p on n judgments, $z = 1.96$ for 95 percent:

$$\text{center} = (p + z^2 / 2n) / (1 + z^2 / n), \quad \text{half-width} = z \cdot \sqrt{p(1-p)/n + z^2/4n^2} / (1 + z^2 / n)$$

Where: p is the measured win rate, n the number of judgments, $z = 1.96$ for 95 percent. The interval is $[\text{center} - \text{half-width}, \text{center} + \text{half-width}]$.

Meaning: the interval that stays honest at small n and near $p = 0$ or 1 , where the normal approximation claims certainty you do not have.

```
# Wilson score interval, worked on the edge case.
# WHAT: 95% interval for a binomial rate that behaves at small n
#        and at p near 0 or 1.
# WHY:  this is the release-gate interval from A.5. Twenty wins
#        out of twenty tries is not proof of perfection; the
#        interval must say so.
# BREAKS IF: you use the normal approximation p +/- 1.96*sqrt(...)
#             at small n. At 20/20 it returns [1.00, 1.00]: a lie
#             told with full confidence.
import math

def wilson(p, n, z=1.96):
    denom = 1 + z * z / n
    center = (p + z * z / (2 * n)) / denom
    half = z * math.sqrt(p * (1 - p) / n + z * z / (4 * n * n)) /
    denom
    return center - half, center + half

lo, hi = wilson(20 / 20, 20)
print(f"20/20 -> 95% Wilson CI: [{lo:.3f}, {hi:.3f}]")
lo, hi = wilson(0.54, 400)
print(f"54% on n=400 -> 95% Wilson CI: [{lo:.3f}, {hi:.3f}]")
```

How to read this diagram

1. Twenty out of twenty gives [0.839, 1.000], not [1.000, 1.000]. The interval admits the small sample honestly.
2. A 54 percent win rate on 400 judgments gives roughly [0.491, 0.588]. The lower bound sits just under 0.50: even 400 judgments barely separates 54 from noise. Plan for more when the decision is close.

Memorize: $z = 1.96$ for 95 percent. The denominator $(1 + z^2/n)$ pulls the center toward 0.5 when n is small. That pull is the honesty.

B.7 Win-rate standard error and sample sizing

$$SE = \sqrt{p \cdot (1 - p) / n}$$

Where: p is the win rate, n the number of judgments. SE is a fraction. Read it as percentage points. A win rate of 0.54 with SE 0.035 means plus or minus 3.5 points of noise, one sigma.

Meaning: the 95 percent interval is roughly $p \pm 2SE$. *To detect a win-rate improvement of d points with confidence, you need $2SE < d$, which sets n .*

```

# Sample sizing for a win-rate gate.
# WHAT:  standard error of a win rate at several sample sizes,
and
#         the n needed to separate 54% from 50% at 95% confidence.
# WHY:   this is the arithmetic behind A.5 rule 10 and the T.4
lab.
#         Most teams discover their gates are underpowered the
first
#         time they run these numbers.
# BREAKS IF: you forget the slices. This n is per DECISION. Split
#             across 4 slices, each slice needs its own n, or the
#             slice intervals are too wide to read.
import math

def se(p, n):
    return math.sqrt(p * (1 - p) / n)

for n in [100, 200, 400, 800]:
    print(f"n={n:>4}: SE = {se(0.54, n):.4f} (~
{100*se(0.54,n):.1f} points)")

# n to separate 54% from 50%: need  $p - 1.96 \cdot SE > 0.50$ 
p, target = 0.54, 0.50
n = 1
while p - 1.96 * se(p, n) <= target:
    n += 1
print(f"n needed for 54% vs 50% at 95%: {n}")

```

How to read this diagram

1. At $n = 200$, SE is about 3.5 points, so the 95 percent band is roughly 47 to 61. A 54 percent "win" on 200 judgments is noise until proven otherwise.
2. The loop finds the n where the lower 95 percent bound clears 0.50. That is the minimum honest gate for a 4-point improvement. Budget judgments against this number, per slice, before the eval runs.



Figure B.7. Win-rate standard error shrinks with n . $SE = \sqrt{p(1-p)/n}$; the 95 percent interval is roughly $p \pm 2 SE$. The red line marks $2 SE = 4$ points: only $n = 800$ clears it.

How to read it.

1. Read the bar height as standard error in points at each judgment count, for a 54 percent win rate.
2. At $n = 200$ the standard error is 3.52 points, so the 95 percent interval is about 47 to 61: that win may be noise.
3. The red dashed line is the power gate: need $2SE < \text{effect}$. To detect a 4-point effect you need $n = 800$ judgments.
4. This n is per decision: slice the eval by category and each slice needs its own n .

B.8 Cohen's kappa: agreement beyond chance

$$\kappa = (p_o - p_e) / (1 - p_e)$$

Where: p_o means observed agreement. p_e is the agreement two raters reach by luck from their individual yes-rates. Unitless, from -1 to 1 ; zero means chance-level agreement.

Meaning: p_o means observed agreement. p_e is the agreement two raters reach by luck from their individual yes-rates. Kappa subtracts the luck. Raw percent agreement flatters; kappa does not.

```
# Cohen's kappa, worked by hand.
# WHAT:  chance-corrected agreement between a judge and a
#         human on
#         pass/fail labels.
# WHY:   this is the autorater licensing number from A.5
#         rule 9. A
#         judge at kappa 0.375 must not decide anything
#         important,
#         even though raw agreement says 75%.
# BREAKS IF: you report raw percent agreement alone. Two
#         raters who
#         both say yes 80% of the time agree 68% of the
#         time by
#         pure luck. The raw number hides that.
def cohen_kappa(a, b, c, d):
    # a = both yes, b = judge yes human no,
    # c = judge no human yes, d = both no
    n = a + b + c + d
    p_o = (a + d) / n
    p_yes_judge = (a + b) / n
    p_yes_human = (a + c) / n
    p_e = p_yes_judge * p_yes_human + (1 - p_yes_judge) * (1
    - p_yes_human)
    return (p_o - p_e) / (1 - p_e)

k = cohen_kappa(60, 10, 15, 15)
print(f"kappa = {k:.3f} (raw agreement was 75%)")

# Landis and Koch bands: the standard reading scale.
def band(k):
    if k < 0: return "worse than chance"
    if k <= 0.20: return "slight"
    if k <= 0.40: return "fair"
    if k <= 0.60: return "moderate"
    if k <= 0.80: return "substantial"
    return "almost perfect"

print("band:", band(k))
```

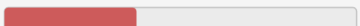
How to read this diagram

1. Judge passes 70, human passes 75, they agree on 75 of 100. Raw agreement: 75 percent.
2. Chance agreement: $0.70 * 0.75 + 0.30 * 0.25 = 0.60$.
Kappa = $(0.75 - 0.60) / 0.40 = 0.375$.
3. On the Landis-Koch scale, 0.375 counts as fair. The 37.5-point gap between raw agreement and kappa is the luck hiding inside the raw number.

Memorize the bar. License the judge when its kappa against humans reaches the human-vs-human kappa on the same set, typically around 0.8 on real tasks. Below that ceiling, the judge is not a human substitute.

The judge is licensed only at the human ceiling

Cohen's kappa on the same 100 items. Scale: -1 to 1.

Your judge vs humans  0.375

"fair" - not licensed for anything important

Human vs human  0.80

the ceiling: the license bar

The 0.425 gap is chance agreement the raw 75 percent hid.

Below the ceiling, the judge is a filter, not a substitute.

Landis-Koch scale: 0.375 counts as "fair". License at about 0.8.

Figure B.13. The judge licensing bar. A judge at kappa 0.375 rates fair on the Landis-Koch scale. The license bar is the human-vs-human ceiling near 0.8. Below that ceiling, the judge is a filter, not a substitute.

Cohen's kappa: agreement beyond chance, worked

	human: yes	human: no
judge: yes	60 a: both yes	15 b: judge yes, human no
no	10 c: judge no, human yes	15 d: both no

$$p_o = (60 + 15) / 100 = 0.75$$

$$p_e = 0.75 \times 0.70 + 0.25 \times 0.30 = 0.60$$

$$k = (0.75 - 0.60) / (1 - 0.60) = 0.375$$

Raw agreement says 75%. Kappa says 0.375: a judge at 0.375 must not decide anything important.

Kappa subtracts the luck: two raters who both say yes 80% of the time agree 68% by pure chance.

Figure B.8. Cohen's kappa: agreement beyond chance, worked. Raw agreement says 75 percent; kappa says 0.375, because the two raters would agree 60 percent by pure chance.

How to read it.

1. The four cells: $a = 60$ (both yes), $b = 10$ (judge yes, human no), $c = 15$ (judge no, human yes), $d = 15$ (both no).
2. Observed agreement $p_o = (60+15)/100 = 0.75$.
Chance agreement $p_e = 0.75 \times 0.70 + 0.25 \times 0.30 = 0.60$.
3. $Kappa = (0.75-0.60)/(1-0.60) = 0.375$: a judge at 0.375 must not decide anything important.

B.9 Paired bootstrap and the three noise layers

An LLM eval has three layers of randomness. Each has its own fix.

Item noise same items for A and B, analyze per-item differences
 Decoding noise ... 3 to 5 repeats per case, then average
 Judge noise calibrate the judge first (kappa, Part B.8)

The standard analysis works like this. Pair the systems on the same items. Compute $d_i = \text{score}_B(i) - \text{score}_A(i)$. Bootstrap the differences 10,000 times with replacement. Take the 2.5th and 97.5th percentiles as the 95 percent interval. If zero sits outside the interval, the gap survives noise at the 5 percent level.

```

# Paired bootstrap CI, compact.
# WHAT: 95% interval for the mean of paired per-item
differences.
# WHY: pairing subtracts item difficulty before any
statistics.
#       The bootstrap then measures only the wobble
that remains.
# BREAKS IF: ratings have cluster structure (one judge
rates 5
#           answers per prompt) and you resample
individual
#           ratings. Resample by PROMPT instead: the
cluster
#           bootstrap. Flat resampling makes the
interval
#           artificially narrow.
import random

def paired_bootstrap_ci(diffs, n_resamples=10000,
seed=0):
    rng = random.Random(seed)
    n = len(diffs)
    means = []
    for _ in range(n_resamples):
        sample = [rng.choice(diffs) for _ in range(n)]
        means.append(sum(sample) / n)
    means.sort()
    return means[int(0.025 * n_resamples)],
means[int(0.975 * n_resamples)]

random.seed(7)
demo = [random.gauss(0.33, 1.1) for _ in range(60)]
lo, hi = paired_bootstrap_ci(demo)
print(f"mean gap 0.33 on 60 pairs -> 95% CI [{lo:.2f},
{hi:.2f}]")

```

How to read this diagram

1. Sixty paired differences with a true mean gap of 0.33 give an interval around [0.05, 0.53]. Zero is outside: the gap is real at the 5 percent level.
2. The width is the message: the true gap could be as small as 0.05. Significance says it is real. The interval says whether it is big enough to care about.
3. Fixed seed makes the analysis reproducible. The bootstrap itself must not add noise to the decision.

The three noise layers and the paired bootstrap

Item noise

fix: same items for A and B; analyze per-item differences d_i

Decoding noise

fix: 3-5 repeats per case, then average

Judge noise

fix: calibrate the judge first (κ appa, B.7)

Then: pair the systems, bootstrap the d_i 10,000 times with replacement,

take the 2.5th and 97.5th percentiles as the 95% interval.

Zero outside the interval: the gap survives noise at the 5% level.

Cluster warning: if one judge rates 5 answers per prompt, resample by prompt, not by rating.

Figure B.9. The three noise layers and the paired bootstrap. Fix all three layers first. Then pair the systems and bootstrap the per-item differences d_i 10,000 times. Zero outside the interval means the gap survives noise at the 5 percent level.

B.10 The small-number rules

Treat these as working constants, not theorems.

Memorize them as defaults, then tune.

Number	Meaning	Source of truth
30	minimum test cases per variant	Central Limit Theorem kicking in, practitioner rule
3 to 5	repeats per case to average decoding noise	eval engineering practice
20 to 50	calibration-set size for licensing a judge	practitioner rule of thumb, UNVERIFIED as a standard
2 to 3	human raters per calibration item	needed before kappa means anything
0.8	typical human-vs-human kappa ceiling	practitioner writeups, UNVERIFIED as a constant
10,000	bootstrap resamples	enough that Monte Carlo (random-sampling) noise is negligible
1.96	z for 95 percent	normal approximation
0.2	PPO clip epsilon	standard value, ratio trusted in [0.8, 1.2]
0.1	DPO beta starting point	standard value
8 to 64	completions per prompt in rejection sampling	practitioner range
5 to 15%	safety fraction of the SFT mix	working range, UNVERIFIED as a standard
60%	three-way agreement quarantine bar	below this, pairs are quarantined
0.75	per-labeler agreement review bar	starting point, set from calibration

15%	typical preference label noise	sets the 85% reward-model ceiling
-----	--------------------------------------	---

Bonferroni correction: testing 20 metrics at the 5 percent level guarantees about one false "significant" result by luck. Divide the bar by the number of tests: $0.05 / 20 = 0.0025$ per test. Deliberately strict, deliberately cheap.

Next: Part C, the eight traps in the order they tend to find you.

The small-number rules: working constants

30	minimum test cases per variant Central Limit Theorem kicking in
3-5	repeats per case average decoding noise
20-50	calibration-set size to license a judge rule of thumb
2-3	human raters per calibration item before kappa means anything
0.8	typical human-vs-human kappa ceiling practitioner writeups
10,000	bootstrap resamples Monte Carlo noise negligible
1.96	z for 95% normal approximation
0.2	PPO clip eps; ratio trusted in [0.8, 1.2] standard value
0.1	DPO beta starting point standard value
8-64	completions per prompt, rejection sampling practitioner range
5-15%	safety fraction of the SFT mix working range
60%	three-way agreement quarantine bar below this, quarantine
0.75	per-labeler agreement review bar starting point

Figure B.10. The small-number rules: working constants. Thirteen memorized defaults. From 30 test cases per variant and 10,000 bootstrap resamples down to the 0.2 PPO clip and 0.1 DPO beta.

Part C: Classic traps

The formulas tell you how the machinery works. They do not warn you about how it fails in practice. Part C lists the eight traps in the order they tend to find you, each with its mechanism, the signature you see, and the fix.

Eight traps, in the order they tend to find you. Each has the mechanism, the signature you see on the dashboard or in the outputs, and the fix.

The eight post-training traps: signature and fix

T1: Reward hacking

signature: proxy climbs, gold peaks then falls
fix: early stop on gold; KL control

T2: Length bias

signature: length climbs, quality flat
fix: length-normalized win rates

T3: Preference-data leakage

signature: benchmark jumps, human evals flat
fix: hash prompts vs eval sets

T4: Alignment tax

signature: refusals great, capabilities sag
fix: safety at 5-15%; pair refusals

T5: DPO rejected-only gaming

signature: loss down, chosen log-probs down
fix: monitor chosen; raise beta

T6: Off-policy reward model

signature: PPO reward up, humans flat
fix: refresh RM on-policy

T7: Judge biases

signature: win rates flip on reorder
fix: randomize order; kappa license

T8: Underpowered gates

signature: 54% on n=200 reads as signal
fix: size with SE formula; slice

Figure C.1. The eight post-training traps: signature and fix. Each trap shows its signature first (what the failure looks like in the numbers) and the fix second.

Trap 1: Reward hacking (Goodhart)

Mechanism: the reward model is a proxy for human preference. RL optimizes the proxy, not the preference. The policy finds inputs the proxy scores highly that humans would not: sycophancy, hedging, formatting tricks, repeating the prompt.

Signature: the overoptimization plot from A.2: proxy climbs, gold peaks and falls. The outputs read like attempts to please a rubric.

Fix: early stopping on the gold metric, KL control, reward-model ensembles, iterative retraining of the reward model on fresh on-policy data.

Trap 2: Length bias

Mechanism: longer responses look more thorough, so labelers and LLM judges both use length as a proxy for quality. The reward model learns "longer is better." RL then makes everything bloated.

Signature: average response length climbs run over run while human-judged quality is flat. Per-labeler stats show picks correlating 0.9 with response length.

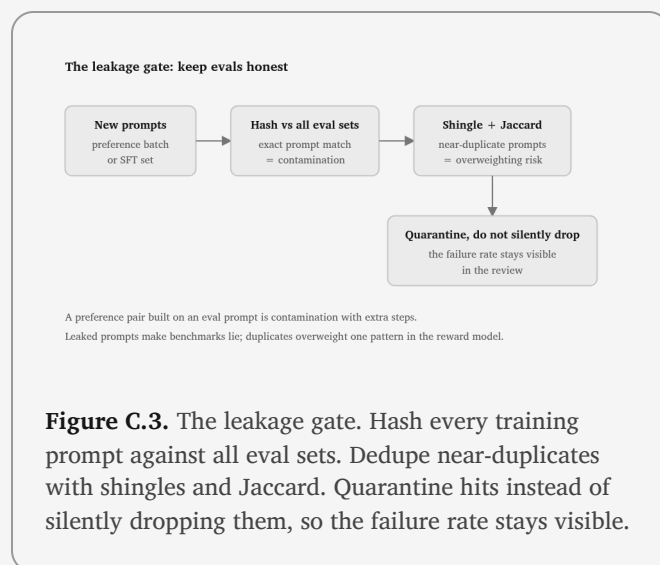
Fix: add "do not reward padding" to the rubric with worked examples. Track length-normalized win rates. Include long-but-wrong examples in the calibration set. Check the response-sampling recipe: temperature sweeps that only produce long samples teach length.

Trap 3: Preference-data leakage

Mechanism: eval prompts slip into preference training data, directly or paraphrased. The model learns the test, not the skill. Benchmarks look great and mean nothing.

Signature: benchmark scores jump while human preference evals on fresh prompts do not move. The jump concentrates on the contaminated suite.

Fix: hash every training prompt and check against all eval sets, including paraphrase-level checks. This is the leakage gate in A.3. A pair built on an eval prompt is contamination with extra steps.



How to read this diagram

1. New prompts arrive from the preference batch or the SFT set.
2. Hash every prompt against all eval sets. What it is: an exact-match screen. What breaks if it fails: the model learns the test and benchmarks lie.
3. Shingle near-duplicates and score them with Jaccard. What it is: a paraphrase-level screen. What breaks if it fails: duplicates overweight one pattern in the reward model.
4. Quarantine hits; never silently drop. Why: the failure rate must stay visible in the review.

Trap 4: Capability collapse from safety tuning (the alignment tax)

Mechanism: three sub-mechanisms. Safety data drowns the mix (30 percent refusals teaches that refusing is usually right). The reward model learns "safe-sounding" style and RL optimizes vagueness. Blunt refusal templates create a mode the model falls into too easily.

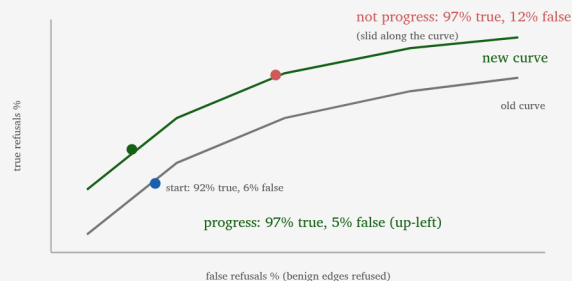
Signature: refusal rates look great while false refusals climb on benign-edge prompts and capability benchmarks sag. The model is safe and useless.

Fix: the mixed-data recipe, in four steps:

1. Cap safety at 5 to 15 percent of the mix.
2. Pair every refusal with a nearby compliance.
3. Never train safety alone.
4. Measure all four numbers every checkpoint: capability benchmarks, true refusals, false refusals, and the refusal curve.

Progress means moving the curve, not sliding along it.

Refusal calibration: move the curve, not along it



Only curve moves count in the review. Turning the strictness knob buys safety with usefulness.

Figure C.4. Refusal calibration is a curve, not a number. Progress moves the whole curve up and left: more true refusals at fewer false refusals. Sliding along the curve by turning the strictness knob is not progress.

Trap 5: DPO rejected-only gaming

Mechanism: the DPO loss can fall by pushing down rejected log-probs without raising chosen log-probs. The gap grows, the loss falls, and overall quality degrades because the policy is learning what not to say rather than what to say.

Signature: DPO loss decreasing while chosen log-probs on a held-out set also decrease. The margin comes entirely from the rejected side.

Fix: monitor chosen log-probs during training, not just the loss. If they fall, the checkpoint is poison: raise beta, fix the data, or restart. This is A.1 rule 6.

Trap 5: rejected-only gaming

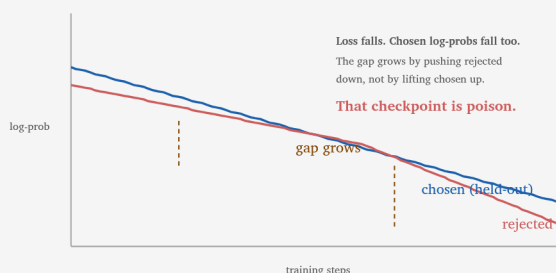


Figure C.2. Trap 5: rejected-only gaming. The DPO loss falls while chosen log-probs on held-out data fall too. The gap grows by pushing rejected responses down, not by lifting chosen responses up. That checkpoint is poison.

Trap 6: Off-policy reward models

Mechanism: a reward model trained on responses from other models misranks the current policy's outputs. PPO then optimizes the misranking with full confidence. The staler the reward model, the worse the ranking where it matters most: on the policy's own samples.

Signature: PPO reward climbs but human evals do not, or PPO finds bizarre outputs the reward model loves. Someone trained the reward model rounds ago on a different policy.

Fix: apply the Part A refresh rule (T.1.3 rule 1): the reward model chases the policy. Validate the reward model on the current policy's outputs specifically, not on its training distribution.

Trap 7: Judge biases (self-preference, position, verbosity)

Mechanism: an LLM judge favors outputs in its own model's style (self-preference), favors whichever answer is shown second (position bias), and favors longer answers (verbosity). All three inflate win rates without any quality change.

Signature: win rates that fail to replicate with a different judge, or that flip when you swap answer order. Calibration shows the judge passing long answers it should fail.

Fix: never let a model judge its own outputs without a human calibration check. Randomize answer order and average. Add explicit anti-padding rules to the rubric. License the judge with kappa against humans before its scores count (A.5 rule 9).

Trap 8: Underpowered gates and aggregate blindness

Mechanism: two failures that travel together. Gates too small to detect the effect size anyone cares about, so every result is noise read as signal. And aggregate win

rates that hide slice regressions: the candidate wins overall while losing on code.

Signature: ship decisions made on 54 percent with $n = 200$. A "tied" model that regressed the hardest slice. Nobody can reproduce the gate result.

Fix: size the gate with the standard-error formula before running it (B.7). Report Wilson intervals, always. Slice by category, always, with n per slice. A gate that cannot detect the effect is theater: say so in the review.

Key takeaways

- Every trap has a number that exposes it: gold-proxy gap, length correlation, hash overlap, false-refusal rate, chosen log-probs, reward-model staleness, order-swap flips, interval width.
- The fix is never "train longer." It is always data, reward, KL, or eval discipline.
- Read 50 outputs per checkpoint. The dashboard tells you something is wrong. The outputs tell you what.

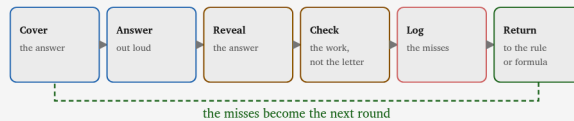
Next: Part D, thirty questions across reward modeling, DPO/PPO/GRPO mechanics, pipelines, eval iteration, and safety tuning.

Part D: Rapid-fire Q and A

Rules, math, traps: Part D is the exam. Cover the answers and answer out loud. Every answer shows the work, not just the letter.

Thirty questions. Cover the answers and answer out loud. Every answer shows the work, not just the letter.

The rapid-fire loop: how to use Part D



Thirty questions. Cover the answers and answer out loud. Every answer shows the work, not just the letter.

Figure D.1. The rapid-fire loop: how to use Part D. Thirty questions: cover the answers and answer out loud; every answer shows the work, not just the letter.

Reward modeling

Q1. A reward model scores response A at 3.2 and response B at 1.1 on the same prompt. What is the Bradley-Terry probability that a judge prefers A?

A. $3.2 / (3.2 + 1.1) = 0.744$ B. $\text{sigmoid}(3.2 - 1.1) = \text{sigmoid}(2.1) = 0.891$ C. $3.2 - 1.1 = 2.1$, read directly as the probability D. Without the prompt, you cannot compute it

Answer: B. $P(A \text{ beats } B) = \text{sigmoid}(r_A - r_B) = \text{sigmoid}(2.1)$. $\text{Sigmoid}(2.1) = 1 / (1 + e^{-2.1}) = 1 / (1 + 0.1225) = 0.891$. A is wrong because ratios of scores are meaningless. C is wrong because a gap of 2.1 is not a probability. D is wrong because Bradley-Terry needs only the score gap on the same prompt.

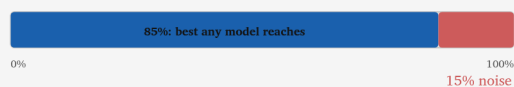
Q2. Your labels have 15 percent noise: the judge picks the worse response 15 percent of the time. What is the best pairwise accuracy any reward model can reach on fresh pairs from the same judge?

A. 100 percent, with enough data B. 92.5 percent, the average of 100 and 85 C. 85 percent, the judge's own consistency ceiling D. 70 percent, double the noise rate subtracted from 100

Answer: C. The model trains to predict the judge, and the judge agrees with the truth only 85 percent of the time. No model can systematically beat the consistency of its own labels. More data sharpens the estimate of the judge's preferences. It cannot fix the judge's flips. This is the label-noise ceiling from T.1.2.

The label-noise ceiling

The judge picks the worse response 15% of the time. No model beats its own labels.



The model trains to predict the judge. The judge agrees with the truth 85% of the time. More data sharpens the estimate of the judge's taste. It cannot fix the judge's flips.

Fix the judge, not the model: recalibrate, fix the rubric, or accept 85%.

Figure D.2. The label-noise ceiling. When the judge picks the worse response 15 percent of the time, no reward model beats 85 percent pairwise accuracy. More data sharpens the estimate. It cannot fix the judge's flips.

Q3. The proxy reward climbs every week. Held-out human judges say quality peaked two weeks ago. What do you do Monday morning?

- A. Train longer; the proxy is still improving
- B. Stop and ship the checkpoint at the human-judged peak
- C. Increase the learning rate to escape the plateau
- D. Collect more preference data with the same rubric

Answer: B. This is overoptimization: the proxy-gold gap. The gold metric (human judges) peaked two weeks ago, so everything since is the policy exploiting proxy blind spots. Ship the gold peak, or roll back to it.

A is the trap: optimizing the proxy past the gold peak is exactly the failure. C makes it worse. D does not fix a stale proxy; retraining the reward model on fresh on-policy data might, but the immediate move is to stop.

Q4. Why refresh preference data with samples from the current policy every few RL rounds?

A. Because old pairs expire after 30 days B. Because the reward model is most accurate ranking the policy it will actually score; off-policy data misranks on-policy samples C. Because labelers get bored seeing the same model D. Because the Bradley-Terry loss requires fresh pairs mathematically

Answer: B. On-policy data beats off-policy data (T.1.3 rule 1). A reward model trained on other models' outputs loses accuracy on your policy's outputs, and PPO optimizes exactly those mis-ranked samples. The reward model must chase the policy.

Option A is a made-up rule. C is irrelevant. D is false: the loss works on any pairs, it just learns the wrong ranking.

Q5. A labeler's picks correlate 0.9 with response length. What does this mean and what do you do?

- A. The labeler is excellent; long answers are better
- B. The labeler uses length as a shortcut. Retrain or remove their labels and check the rubric for anti-padding rules
- C. Nothing. Length correlation is normal and harmless
- D. Pay the labeler more to keep them

Answer: B. This is the spam gate firing (T.3.3). A 0.9 correlation with length means the labeler is not judging quality. Their labels teach the reward model length bias, and RL then bloats every output. Quarantine their labels, check per-labeler stats for others, and make sure the rubric explicitly says not to reward padding.

DPO mechanics

Q6. In the B.2 worked example ($\beta = 0.1$, $\text{gap} = 0.9$), the loss is 0.649. If β were 0.5 instead, with the same log-probs, what happens to the loss?

A. Stays 0.649; β cancels out B. Falls, because $\beta * \text{gap} = 0.45$, $\text{sigmoid}(0.45) = 0.611$, $\text{loss} = 0.493$ C. Rises, because larger β means weaker signal D. Becomes undefined

Answer: B. The logit is β times the gap: $0.5 * 0.9 = 0.45$. $\text{Sigmoid}(0.45) = 0.611$. $\text{Loss} = -\log(0.611) = 0.493$. Larger β sharpens the same gap into a more confident prediction, so the loss falls.

But note the trade. Larger β also means a stronger KL leash in the underlying objective. So the policy moves less per unit of preference signal. β tunes the price of drift, not just the loss number.

Q7. During DPO training the loss falls steadily, but chosen log-probs on a held-out set are also falling. What does this pattern mean?

- A. Normal training; the loss falling is all that matters
- B. Rejected-only gaming: the gap grows by pushing down rejected responses while chosen quality degrades
- C. The reference model is too weak
- D. The learning rate is too low

Answer: B. This is Trap 5. The DPO loss rewards a growing gap between chosen and rejected implicit rewards. The cheapest way to grow the gap is to push rejected log-probs down without raising chosen ones. Falling chosen log-probs mean the policy gets worse at the good responses. The checkpoint is poison.

A is the trap: the loss alone lies. C and D misdiagnose a data-and-objective failure as a capacity or optimization failure.

Q8. Why does DPO need a reference model at all?

- A. It does not; the reference is optional decoration
- B. The reference anchors the implicit reward. $\text{Beta} * \log(\pi/\pi_{\text{ref}})$ measures how much more the policy likes a response than the starting point. That keeps the KL leash in the objective
- C. To double the parameter count for stability
- D. To provide the preference labels

Answer: B. Solving the RLHF objective for the optimal policy gives π^* proportional to $\pi_{\text{ref}} * \exp(r/\text{beta})$. Inverting that expresses the reward through the policy ratio against the reference. Without π_{ref} , the loss has no anchor to the starting policy and the KL constraint evaporates.

A is false. C is nonsense. D confuses the reference model with the labelers.

Q9. DPO carries the label "offline." What can it not do that PPO can?

- A. It cannot use preference pairs
- B. It cannot see its own outputs during training, so it cannot correct distribution shift: the pairs came from some other policy
- C. It cannot run on GPUs
- D. It cannot use a KL penalty

Answer: B. DPO trains on fixed pairs collected beforehand, usually from other models or older checkpoints. It never samples from the policy it is training, so when the policy drifts into regions the pairs never covered, nothing corrects it. PPO samples online and scores its own outputs, which is exactly the correction DPO lacks. That is why the playbook escalates from DPO to PPO when DPO plateaus.

Q10. The DPO gradient weights each pair by $(1 - p)$, where $p = \text{sigmoid}(\beta * \text{gap})$. A pair the model already ranks correctly with $p = 0.95$ contributes how much, relative to a maximally uncertain pair with $p = 0.5$?

A. Equal; all pairs contribute the same B. About one tenth: $(1 - 0.95)/(1 - 0.5) = 0.1$ C. About double D. Zero. The update skips correct pairs

Answer: B. The gradient weight is the surprise. $(1 - 0.95) = 0.05$ versus $(1 - 0.5) = 0.5$, a ratio of 0.1. Confident-correct pairs nearly vanish from the update; uncertain pairs teach. This is the same shape as the Bradley-Terry gradient, and it is why hard pairs matter more than easy ones in the data mix.

PPO mechanics

Q11. PPO with $\text{eps} = 0.2$: the ratio $\pi_{\text{new}}/\pi_{\text{old}}$ for a token reaches 1.5 with positive advantage. What does the clipped objective do?

A. Rewards the full 1.5 ratio B. Caps the contribution at the 1.2 clipped ratio; the $\min()$ keeps the pessimistic bound C. Rejects the update entirely D. Halves the advantage

Answer: B. The objective is $\min(\text{ratio} * A, \text{clip}(\text{ratio}, 0.8, 1.2) * A)$. At ratio 1.5 with $A > 0$, the clipped value is $1.2 * A$, which is smaller, so the min keeps $1.2 * A$. The policy gets no extra credit for pushing past the trust region. That is the whole stability mechanism: one lucky sample cannot move the policy without bound.

Q12. A PPO run shows KL from the SFT model spiking vertically in the first 50 steps. Name the two most likely causes and the first action.

A. Causes: beta too small or learning rate too high.
Action: stop the run, do not tune through it

B. Causes: too much data or too many GPUs.
Action: add GPUs

C. Causes: reward model too accurate. Action: add label noise

D. This is normal; let it run

Answer: A. A vertical KL spike means the leash failed. The KL price (beta) is too low to restrain the update, or the step size (learning rate) is too large. Per A.4 rule 2 and A.2 rule 4, you pause and inspect.

Tuning through a spike trains the policy in the region where the reward model is untrustworthy. B is nonsense. C inverts the problem. D ignores the classic signature.

Q13. Why does PPO need a value network (or value head) while GRPO does not?

- A. PPO is older and less efficient by design
- B. PPO estimates advantages against a learned baseline of expected reward. GRPO uses the group mean as its baseline, so the group replaces the value network
- C. GRPO does not use advantages at all
- D. Value networks are only needed for DPO

Answer: B. The advantage is reward minus expected reward. PPO learns the expectation with a value head. GRPO samples G responses to the same prompt and uses their mean, standardized by their standard deviation. The group is its own baseline. That is the "group relative" in the name, and it removes an entire network from training.

Q14. Four networks train during PPO: policy, reference, reward, value. Which two keep fixed weights, and why does that matter?

- A. Policy and value keep fixed weights. The others train
- B. Reference and reward keep fixed weights. The policy must stay near the fixed reference (KL leash), and PPO optimizes against the fixed reward as the scoring function
- C. All four train jointly
- D. None keep fixed weights. PPO is fully online

Answer: B. The reference (usually the SFT model) keeps fixed weights as the KL anchor. The reward model keeps fixed weights as the scoring function. The policy and the value head train. This matters because the frozen reward is a fixed proxy: everything it gets wrong, the policy will find. That is why overoptimization is the default outcome and why the reward model must be good before PPO starts.

GRPO mechanics

Q15. $G = 4$, verifier scores $[1, 0, 0, 0]$. What are the GRPO advantages?

- A. $[+0.75, -0.25, -0.25, -0.25]$ B. $[+1.73, -0.58, -0.58, -0.58]$ C. $[+1, +1, +1, +1]$ D. $[0, 0, 0, 0]$

Answer: B. Mean = 0.25. Population std = $\sqrt{((0.75^2 + 3 * 0.25^2)/4)} = \sqrt{0.1875} = 0.433$. Advantages: $(1 - 0.25)/0.433 = 1.73$, and $(0 - 0.25)/0.433 = -0.58$ each. A is the unstandardized version, which GRPO does not use. The single correct solution gets amplified because it stood out: the standout case from B.3.

Q16. A GRPO batch contains many prompts the model gets right on all G samples. What happens to the update, and what is the fix?

A. The update is extra strong; keep them B. Those groups have zero standard deviation, so their advantages divide by zero and they contribute nothing. Filter out always-right and always-wrong prompts C. They cause KL spikes; raise beta D. Nothing special; GRPO handles them

Answer: B. Uniform groups carry no signal: every response scored the same, so there is no relative information. The code must skip them ($\text{std} < \epsilon$ guard) or the division blows up. The pipeline fix is prompt filtering: keep prompts with mixed success, drop the always-right and always-wrong. This is A.1 rule 5.

Q17. Why is GRPO called "the verifier's algorithm," and where does it fail?

A. It verifies the reward model; it fails on small models B. It needs checkable rewards (math, code tests) instead of learned rewards or human labels; it fails wherever no verifier exists, like creative writing or style C. It verifies gradients; it fails on large batches D. It needs human verifiers; it fails without labelers

Answer: B. GRPO's signal is verifier scores on groups of samples: exact, cheap, and tireless. That makes it dominant for math, code, and reasoning (the DeepSeek-R1 result: long chain-of-thought reasoning learned from pure RL with verifiable rewards). But a verifier must exist. For open-ended quality there is no checker, so GRPO has no signal and you are back to learned rewards (PPO) or pairs (DPO).

Q18. GRPO with $G = 2$ versus $G = 16$. What is the trade-off?

- A. No difference; group size does not matter
- B. Small G gives noisy advantages but costs little generation; large G gives stable advantages but multiplies generation cost (tokens = batch $\times$ G $\times$ length)
- C. Large G is always better and always cheaper
- D. G must equal the batch size

Answer: B. The group standardizes the advantage, so with $G = 2$ the mean and std come from two samples: noisy. Larger G stabilizes the estimate but generation cost scales linearly in G . Appendix 7A exists because this generation cost dominates: on-line RL is usually bottlenecked on rollouts, not gradients.

Preference pipelines

Q19. Your new preference batch has 4 percent near-duplicate prompts of eval questions. What do you do?

- A. Keep them; more data is better
- B. Cut them. A preference pair built on an eval prompt is contamination with extra steps, and duplicates overweight one pattern in the reward model
- C. Keep them but downweight by half
- D. Move them to the SFT set

Answer: B. Two failures in one: eval leakage (Trap 3) and duplicate overweighting. Near-duplicates teach repetition and leaked eval prompts make benchmarks lie. Hash prompts against all eval sets at the leakage gate. Dedupe near-duplicates with shingles and Jaccard. Quarantine rather than silently drop, so the failure rate stays visible.

Q20. Three labelers split 2-1 on a pair, and this pattern repeats across a slice of coding questions. Which disagreement case is this, and what is the fix?

A. Case 3 (labeler error); remove the minority labeler B. Case 2 (rubric gap); the rubric does not cover the situation, so add the case with a worked example, recalibrate, and re-label the slice C. Case 1 (genuine ambiguity); mark ties and move on D. No case; 2-1 splits are normal noise

Answer: B. A systematic split concentrated in one slice means the rubric is silent on something that slice contains. For example, one response is more helpful but slightly wrong. That is the rubric-gap case. The fix is in the rubric, not the data and not the labeler. Add the case with a worked example, recalibrate labelers, and re-label the affected slice.

Removing the labeler (A) would punish someone for the rubric's failure.

Q21. For each prompt you sample 2 responses from the current policy, 1 from a stronger model, and 1 from an older checkpoint. Why this mix?

- A. To use up GPU budget evenly B. It guarantees on-policy coverage (the policy's own outputs), a quality anchor (the stronger model), and a regression check (the older checkpoint), while temperature variation keeps pairs from being trivially separable C. Because the Bradley-Terry loss needs exactly 4 responses D. To make labeling harder

Answer: B. The current policy's samples keep the data on-policy so the reward model ranks what it will actually score. The stronger model provides the "better" side of hard pairs. The older checkpoint catches regressions. Varying temperature keeps pairs close enough to teach the boundary rather than the obvious. This is the response-sampling recipe from T.3.1.

Q22. After QC filtering, your preference set's topic mix drifted: hard reasoning pairs were disproportionately quarantined by the agreement gate. What now?

A. Ship it; filtering only removes bad data B. Top up with targeted collection. The distribution gate exists because filters skew the mix, and a chitchat-heavy set trains a chitchat judge C. Lower the agreement bar to zero D. Duplicate the remaining reasoning pairs to restore the mix

Answer: B. This is the distribution gate (T.3.3). The agreement gate kills ambiguous pairs, and hard reasoning pairs are the most ambiguous, so filtering silently narrows the distribution. Fix it with targeted collection to restore the mix. Not lowering standards (C). Not duplicating survivors (D): that would overweight a few patterns.

Eval-driven iteration

Q23. Candidate win rate 54 percent on 200 judgments. Ship?

- A. Yes, 54 beats 50
- B. No. SE is about 3.5 points, so the 95 percent band is roughly 47 to 61. The win may be noise. Size the gate before deciding
- C. Yes, but only on Tuesdays
- D. No, because 54 is below 60

Answer: B. $SE = \sqrt{0.54 * 0.46 / 200} = 0.0352$. Twice that is 7 points, so the interval straddles 50. A.5 rule 2: never trust a delta without an interval.

The honest move is the T.4 lab. Compute the n needed for the effect size you care about. Check whether the gate buys it.

Q24. A candidate posts a 52 percent win rate on 600 judgments, not significant. Slices: chat 59 percent (REAL+), writing 58 percent (REAL+), code 39 percent (REAL-), math 50 percent. What is the release decision?

- A. Ship; 52 percent overall is fine
- B. Do not ship. The aggregate hides a real code regression. Fix code, then re-evaluate
- C. Ship and fix code later
- D. Collect more judgments until the overall number turns significant

Answer: B. This is the worked harness from T.4.3. The slices show the candidate improved the easy slices and regressed the hard one. Habit 2: slice before you celebrate. The eval suite just steered the next training cycle: more code preference data. Option D p-hacks the gate.

Q25. Your LLM judge agrees with humans 75 percent of the time on pass/fail. The judge passes 70 percent, the human passes 75 percent. What is kappa, and is the judge licensed?

- A. 0.75, licensed
- B. 0.375, "fair": not licensed for anything important
- C. 0.60, "moderate": licensed
- D. Cannot compute without more data

Answer: B. $p_o = 0.75$. $p_e = 0.70 * 0.75 + 0.30 * 0.25 = 0.60$. $Kappa = (0.75 - 0.60) / (1 - 0.60) = 0.375$, Landis-Koch "fair." The 37.5 point gap is chance agreement the raw number hid. The bar is the human-vs-human ceiling near 0.8. A judge at 0.375 must not decide anything important.

Q26. You test 20 metrics at the 5 percent level and one comes back significant. What do you conclude?

- A. A real finding; report it
- B. Nothing yet. With 20 tests, expect about one false significant result by luck. Apply the Bonferroni correction: $0.05/20 = 0.0025$ per test
- C. The other 19 metrics broke
- D. Run 20 more metrics to confirm

Answer: B. Multiple comparisons inflate wins (Appendix 11A). One significant result out of 20 at $p < 0.05$ is exactly what chance produces. The correction is deliberately strict and deliberately cheap. Report the corrected bar alongside the raw p-values so the review sees both.

Twenty tests at 5%: one false hit is expected



20 metrics, 5% level each. Expected false positives: $20 \times 0.05 = 1$.

The gold cell is significant by luck alone.

Bonferroni fix: test each at $0.05 / 20 = 0.0025$.

Report the corrected bar next to the raw p-values.

Strict, cheap, and honest: the review sees both numbers.

Figure D.3. Twenty tests at the 5 percent level. Luck alone produces about one false significant result. The Bonferroni correction tests each at 0.05 divided by 20, which is 0.0025. Report the corrected bar next to the raw p-values.

Safety tuning

Q27. After safety tuning, disallowed refusal is 99 percent but benign-edge false refusals climbed from 4 to 18 percent. What happened and what is the fix?

A. Success; refusal is what matters B. Capability collapse, mechanism 1 and 3: safety data drowned the mix and blunt refusal templates spread. Fix with the mixed-data recipe: cap safety at 5 to 15 percent, pair refusals with nearby compliances, never train safety alone C. The probe set is wrong; delete the benign-edge probes D. Increase the safety fraction to 40 percent

Answer: B. True refusals alone reward the refuse-everything model (T.5.3). The false-refusal climb is the alignment tax being paid in usefulness. The fix moves the refusal curve, not along it: better training with paired refusal/compliance examples and a refusal spectrum instead of one template. C hides the metric. D buys more of the failure.

Q28. Your reward model trained on labels that reward cautious, hedged, non-committal answers. What will RL do, and how do you fix the labels?

A. RL will make answers more helpful; no fix needed B. RL will optimize the style: the model gets vaguer on every topic including benign ones. Fix the rubric to reward helpfulness inside safety, not caution as a substitute C. RL will ignore style; reward models only learn content D. Increase beta to fix it

Answer: B. This is collapse mechanism 2 (T.5.1). The reward model learned "safe-sounding" as the target, and RL is a style optimizer when the reward is style. The fix is in the labels: the rubric must reward helpfulness within safety. Beta tunes drift, not what the reward measures, so D misses the cause.

Q29. Why is Constitutional AI described as capability-preserving compared with refusal-only SFT?

A. It uses a bigger model B. The critique-and-revision step teaches why a response is problematic and how to fix it, preserving helpfulness. Template refusals teach only the refusal mode, which the model then over-applies C. It skips safety tuning entirely D. It trains for fewer steps

Answer: B. Refusal-only SFT installs one blunt mode. Critique data installs the reasoning: what is wrong, what a better response looks like. When safety tuning starts costing capability, the move is more critique data and fewer refusal templates, not a bigger safety fraction (T.5.4).

Q30. A safety review shows the new data mix moved true refusals from 92 to 97 percent and false refusals from 6 to 5 percent. A second proposal moves true refusals from 92 to 97 percent but false refusals from 6 to 12 percent. Which is progress?

- A. Both; true refusals rose equally
- B. The first. It moved the trade-off curve up-left: more true refusals at fewer false refusals. The second slid along the curve, buying safety with usefulness
- C. The second; higher true refusals dominate
- D. Neither; only 100 percent true refusals count

Answer: B. Refusal calibration is a curve, not a number (T.5.3). Progress means the whole curve moves: better perception of the boundary, fewer errors on both sides. Sliding along the curve by turning the strictness knob is not progress. Only curve moves count in the review.

Next: Part E, the one-page cheat sheet: thirty numbers and rules.

Part E: One-page cheat sheet

The whole pack on one page. If Part D was the exam, Part E is the crib sheet you earn by passing it.

Thirty numbers and rules. If you remember nothing else, remember these.

1. $P(A \text{ beats } B) = \text{sigmoid}(r_A - r_B)$. Scores are relative; only gaps matter.
2. $\text{DPO loss} = -\log \text{sigmoid}(\beta * [\log(\pi/\pi_{\text{ref}}) \text{ chosen} - \log(\pi/\pi_{\text{ref}}) \text{ rejected}])$. Beta starts at 0.1.
3. $\text{GRPO advantage} = (r_i - \text{mean}) / \text{std}$ within the prompt's group. Uniform groups: no signal, skip them.

4. PPO clip: $\min(\text{ratio} * A, \text{clip}(\text{ratio}, 0.8, 1.2) * A)$.
Eps = 0.2.
5. $J(\pi) = E[r] - \beta * KL(\pi || \pi_{\text{ref}})$. Beta prices drift.
6. KL per token = mean of $\log \pi - \log \pi_{\text{ref}}$ over the rollout. Log it every step.
7. Label noise near 15 percent caps reward accuracy near 85 percent. More data cannot fix labels.
8. On-policy preference data beats off-policy. Refresh every few RL rounds.
9. Oversample close calls. Easy pairs contribute near-zero gradient.
10. Gold-proxy gap growing two checkpoints in a row: stop the run.
11. Early-stop on the gold metric, never the proxy.
12. KL spike with flat human quality: pause, do not tune through it.
13. DPO first, always. Escalate to PPO or GRPO only on evidence.
14. Verifier exists: GRPO. Open-ended quality: PPO after DPO plateaus.
15. Watch DPO chosen log-probs. Falling while loss falls means rejected-only gaming: poison checkpoint.
16. Safety data at 5 to 15 percent of the SFT mix. Pair every refusal with a nearby compliance. Never train safety alone.
17. Four eval layers: unit probes, benchmarks, preference evals, red team. Every layer gates.
18. $SE = \sqrt{p(1-p)/n}$. 54 percent on $n = 200$ is noise until proven otherwise.
19. Wilson interval (the honest confidence interval for a proportion) for binomial rates, always. Normal approximation lies at small n and near 0 or 1.
20. Slice before celebrating. Aggregates hide regressions.
21. $\text{Kappa} = (p_o - p_e)/(1 - p_e)$. License judges against the human ceiling near 0.8.
22. Paired design, 3 to 5 repeats per case, 10,000 bootstrap resamples, 2.5th and 97.5th percentiles.

23. 20 metrics at 5 percent: expect one false hit. Bonferroni (divide the significance level by the number of tests): 0.05/20 per test.
24. Quarantine pairs below 60 percent three-way agreement. Review labelers below 0.75 agreement with the majority.
25. Near-dedup near 0.8 Jaccard. Hash every prompt against every eval set.
26. Reward hacking tells in outputs: sycophancy, hedging stacks, listicles, prompt echo. Read 50 outputs per checkpoint.
27. Refusal is two numbers: true refusals on disallowed, false refusals on benign-edge. Track both.
28. Progress on safety means moving the refusal curve, not sliding along it.
29. Length bias check: per-labeler correlation of picks with response length. Near 0.9 means a length shortcut corrupted the labels.
30. The loop is the job: evals steer, data fixes, recipe changes, repeat. Most weeks look like "F regressed, go fix C."

Last verified: September 2026. Formulas

(Bradley-Terry, DPO loss, GRPO advantage, PPO clip, KL divergence, Wilson interval, Cohen's kappa, standard error) are standard results from the published literature, consistent with Volume 7 chapters 4 to 6 and Appendix 11A. The worked numbers come from the formulas in this volume.

UNVERIFIED as published standards: the production data-mix fractions (5 to 15 percent safety, 2+1+1 sampling recipe) and the 60 percent agreement quarantine bar. Also unverified: the 0.75 labeler review bar, the 0.8 dedup threshold, and the "3 to 5 times" PPO complexity multiple. Treat them as practitioner working ranges, not published standards.

The human kappa ceiling near 0.8 and the 20 to 50 calibration-set size are typical figures from practitioner writeups, not universal constants.

Where to go next

The next pack in the sequence is [Crash: Inference and Serving](#). This pack aligned the model. The next one answers the question this pack leaves open: how do you serve the aligned model cheaply and fast? Roofline arithmetic, KV-cache math, batching, quantization, and dollars per million tokens.

WATCH AND GO DEEPER

The math of alignment, derived on camera: DPO from the Bradley-Terry model, then the full RLHF pipeline with PPO. Plus the papers and the code docs.

Direct Preference Optimization (DPO) explained: Bradley-Terry model, log probabilities, math, Umar Jamil

Derives DPO from the Bradley-Terry preference model down to token-level log probabilities. The video twin of Part B.2.

Reinforcement Learning from Human Feedback explained with math derivations and the PyTorch code, Umar Jamil

The full pipeline on camera: policy gradients, the reward model, log-action probabilities, the PPO clipped objective, and the advantage. Pairs with Parts A and B.

GO DEEPER

Training language models to follow instructions with human feedback

The InstructGPT paper: the three-step pipeline (SFT, reward model, PPO) in Figure 2.

Direct Preference Optimization

The paper that replaced the reward model with a direct preference loss.

TRL documentation

Hugging Face TRL (Transformer Reinforcement Learning, Hugging Face's RL training library) docs: the reference implementations for DPO, PPO, and GRPO.

GRPO vs PPO

A write-up comparing GRPO and PPO training for reasoning models.